

## 1. Cover Page

**[TO FILL: AFFILIATED TO UNIVERSITY NAME]**

**ACADEMIC YEAR: 2025–2026**

## 2. Certificate

**[TO FILL: FULL NAME OF COLLEGE / INSTITUTION]**

**[TO FILL: Department of Computer Science / Information Technology]**

**[TO FILL: College Address, City, State - Pincode]**

## CERTIFICATE OF APPROVAL

This is to certify that the project entitled "**Brewline Cafe: A Cloud-Native Artisanal Cafe Ordering, Real-Time Lifecycle Tracking, and Kitchen Operations Platform**" is a bonafide work carried out and successfully completed by:

**[TO FILL: FULL NAME OF STUDENT]**

**Roll No:** [TO FILL: ROLL NO]      **Seat No:** [TO FILL: SEAT NO]

in partial fulfillment of the requirements for the award of the Degree of **[TO FILL: Bachelor of Science in Computer Science / Information Technology (B.Sc. CS / IT)]** of **[TO FILL: Name of the University]** during the Academic Year **2025–2026**.

It is further certified that the candidate has completed all required technical investigations, system design, architectural modeling, coding, and testing under our direct supervision and guidance, and that no part of this report has been previously submitted for any other degree, diploma, or certificate.

[TO FILL: Guide Name]

Project Guide / Supervisor

[TO FILL: Coordinator Name]

## Project Coordinator

---

[TO FILL: HOD Name]

Head of Department

---

Internal Examiner

Date: \_\_\_\_/\_\_\_\_/2026

College Seal

---

External Examiner

Date: \_\_\_\_/\_\_\_\_/2026

---

### 3. Declaration

I, the undersigned, hereby declare that the final-year project report entitled "**Brewline Cafe: A Cloud-Native Artisanal Cafe Ordering, Real-Time Lifecycle Tracking, and Kitchen Operations Platform**" submitted to [TO FILL: Name of College], affiliated with [TO FILL: Name of University], is an original and authentic record of my own work carried out under the academic supervision and guidance of [TO FILL: Prof. Guide Name], Department of [TO FILL: Department Name].

I solemnly affirm that:

- The codebase, architectural design, database schemas, API integrations, and validation testing presented herein were developed by me based on the real-world operational workflow of an artisanal bakehouse and specialty cafe.
- The project has not formed the basis for the award of any other Degree, Diploma, Fellowship, Associateship, or similar academic title at this or any other institution of higher learning.
- All relevant research literature, external open-source libraries, cloud APIs, software frameworks, and documentation consulted during the course of this development have been duly acknowledged and cited in the References section.
- I have strictly abided by the academic integrity and anti-plagiarism guidelines established by the institution and university.

**Place:** [TO FILL: City, e.g., Mumbai / Pune]

**Date:** [TO FILL: DD/MM/2026]

---

[TO FILL: FULL NAME OF STUDENT]

Roll No: [TO FILL: ROLL NO]

Seat No: [TO FILL: SEAT NO]

Class: [TO FILL: T.Y.B.Sc. CS / IT, Semester VI]

---

## 4. Acknowledgement

The completion of this final-year undergraduate project report marks a significant milestone in my academic and technical journey. It is a privilege to express my deepest gratitude to all individuals and organizations whose constant mentorship, technical critique, and encouragement brought this project to fruition.

First and foremost, I wish to convey my profound gratitude and indebtedness to my esteemed project guide, **[TO FILL: Prof. Guide Name]**, [TO FILL: Designation], Department of [TO FILL: Department Name], for invaluable guidance, patient mentoring, rigorous code reviews, and constant encouragement throughout the lifecycle of this project. Their constructive feedback on system modeling, real-time database architecture, and project documentation was instrumental in refining the platform from conceptual inception to full realization.

I extend my sincere thanks to **[TO FILL: Prof. HOD Name]**, Head of the Department of [TO FILL: Department Name], for fostering an intellectually stimulating academic environment and providing state-of-the-art laboratory computing infrastructure and high-speed network connectivity necessary for testing cloud-hosted applications.

I express my respectful appreciation to our respected Principal, **[TO FILL: Dr. Principal Name]**, for leadership, continuous institutional support, and providing all essential facilities within **[TO FILL: Name of College]**.

I also extend my heartfelt appreciation to all faculty members, teaching staff, and technical laboratory assistants of the Department of [TO FILL: Department Name] for their direct and indirect technical assistance, foundational coursework instructions, and helpful suggestions during academic project reviews.

Special gratitude is extended to the open-source software community and cloud technology providers—specifically the engineering teams behind Next.js, React, Google Firebase, Tailwind CSS, Razorpay, and Nodemailer—whose reliable toolchains and libraries made the realization of this web platform possible.

Finally, I express my warmest thanks and eternal debt to my parents, family members, and colleagues for their unwavering moral encouragement, endless patience, and constant sacrifices during the demanding weeks of development, testing, and report preparation.

[TO FILL: FULL NAME OF STUDENT]

Roll No: [TO FILL: ROLL NO]

Seat No: [TO FILL: SEAT NO]

---

## 5. Table of Contents

| Section / Chapter | Title                                | Page No.  |
|-------------------|--------------------------------------|-----------|
|                   | <b>Front Matter</b>                  |           |
|                   | Cover Page                           | i         |
|                   | Certificate of Approval              | ii        |
|                   | Declaration                          | iii       |
|                   | Acknowledgement                      | iv        |
|                   | Table of Contents                    | v         |
|                   | Table of Figures                     | vii       |
|                   | Project Synopsis                     | viii      |
|                   | Plagiarism Verification Report       | x         |
| <b>Chapter 1</b>  | <b>Introduction</b>                  | <b>1</b>  |
| 1.1               | Background of the Study              | 1         |
| 1.2               | Objectives of the Project            | 3         |
| 1.3               | Purpose, Scope, and Applicability    | 5         |
| 1.3.1             | Purpose                              | 5         |
| 1.3.2             | Scope (In-Scope & Out-of-Scope)      | 6         |
| 1.3.3             | Applicability & Target Users         | 8         |
| 1.4               | Technical Achievements of the System | 9         |
| 1.5               | Project Planning & Gantt Chart       | 11        |
| <b>Chapter 2</b>  | <b>Requirements and Analysis</b>     | <b>14</b> |
| 2.1               | Problem Definition                   | 14        |
| 2.1.1             | Identified Operational Bottlenecks   | 14        |
| 2.1.2             | Impact on Key Stakeholders           | 16        |
| 2.1.3             | Proposed Engineering Solution        | 17        |
| 2.2               | Requirement Specification            | 18        |
| 2.2.1             | Functional Requirements (FR-01 to    | 18        |



|                  |                                                           |           |
|------------------|-----------------------------------------------------------|-----------|
|                  | FR-12)                                                    |           |
| 2.2.2            | Non-Functional Requirements<br>(NFR-01 to NFR-08)         | 22        |
| 2.3              | Planning, Scheduling, and<br>Milestones                   | 25        |
| 2.4              | Software and Hardware<br>Requirements                     | 28        |
| 2.4.1            | Hardware Requirements<br>(Development & Production)       | 28        |
| 2.4.2            | Software Environment &<br>Dependency Specifications       | 29        |
| <b>Chapter 3</b> | <b>System Design</b>                                      | <b>31</b> |
| 3.1              | Basic System Modules                                      | 31        |
| 3.1.1            | Customer Ordering Subsystem                               | 31        |
| 3.1.2            | Backend & Cloud Services Engine                           | 33        |
| 3.1.3            | Admin & Kitchen Operations<br>Subsystem                   | 34        |
| 3.2              | Data Design                                               | 36        |
| 3.2.1            | Cloud Firestore Schema Design &<br>Field Specifications   | 36        |
| 3.2.2            | Entity-Relationship (ER) Modeling                         | 40        |
| 3.2.3            | Data Integrity, Snapshot<br>Immutability, and Constraints | 42        |
| 3.3              | System Logic Diagrams                                     | 44        |
| 3.3.1            | System Event Table                                        | 44        |
| 3.3.2            | Use Case Diagram & Actor<br>Descriptions                  | 46        |
| 3.3.3            | Activity Diagrams                                         | 48        |
| 3.3.4            | Class Diagram                                             | 50        |
| 3.3.5            | Object Diagram                                            | 52        |
| 3.3.6            | Sequence Diagrams                                         | 54        |
| 3.3.7            | State Transition Chart                                    | 56        |
| 3.3.8            | Component Diagram                                         | 58        |
| 3.3.9            | Deployment Diagram                                        | 60        |
| 3.4              | User Interface Design & Visual                            | 62        |

|                  |                                                       |            |
|------------------|-------------------------------------------------------|------------|
|                  | System                                                |            |
| 3.4.1            | Design System Tokens & Color Palette                  | 62         |
| 3.4.2            | Detailed Screen Descriptions & Wireframe Architecture | 64         |
| 3.5              | Test Case Design Strategy                             | 68         |
| <b>Chapter 4</b> | <b>Implementation and Testing</b>                     | <b>70</b>  |
| 4.1              | Key Source Code Implementation                        | 70         |
| 4.1.1            | Centralized Pricing & Tax Computation Engine          | 70         |
| 4.1.2            | Server-Side Razorpay Payment Route                    | 72         |
| 4.1.3            | Automated HTML Digital Invoice Generation             | 74         |
| 4.1.4            | Real-Time Firestore Stepper Listener                  | 76         |
| 4.1.5            | Administrative Menu Inventory Stock Controller        | 78         |
| 4.2              | Testing Approach & Methodologies                      | 80         |
| 4.2.1            | Unit Testing                                          | 80         |
| 4.2.2            | Integration & API Testing                             | 82         |
| 4.2.3            | User Acceptance & Beta Testing                        | 84         |
| 4.3              | Comprehensive Test Cases Execution Table              | 86         |
| <b>Chapter 5</b> | <b>Conclusion and Future Enhancements</b>             | <b>94</b>  |
| 5.1              | Conclusion & Major Learning Outcomes                  | 94         |
| 5.2              | Limitations of the Current System                     | 96         |
| 5.3              | Future Scope & Roadmap                                | 98         |
|                  | <b>References &amp; Bibliography</b>                  | <b>101</b> |
|                  | <b>Project Glossary</b>                               | <b>103</b> |

---

## 6. Table of Figures

| Figure No.  | Caption / Description                                         | Chapter   | Page No. |
|-------------|---------------------------------------------------------------|-----------|----------|
| Figure 1.1  | Project Timeline and Phase-Wise Gantt Chart                   | Chapter 1 | 13       |
| Figure 2.1  | System Functional Decomposition Architecture                  | Chapter 2 | 21       |
| Figure 3.1  | Cloud Firestore Entity-Relationship (ER) Schema Diagram       | Chapter 3 | 41       |
| Figure 3.2  | Brewline Cafe System Use Case Diagram                         | Chapter 3 | 47       |
| Figure 3.3  | Customer Checkout and Razorpay Payment Activity Diagram       | Chapter 3 | 49       |
| Figure 3.4  | System Structural Class Diagram                               | Chapter 3 | 51       |
| Figure 3.5  | Runtime Order and User Object Instance Diagram                | Chapter 3 | 53       |
| Figure 3.6  | Real-Time Order Placement and Notification Sequence Diagram   | Chapter 3 | 55       |
| Figure 3.7  | Order Lifecycle State Machine Transition Diagram              | Chapter 3 | 57       |
| Figure 3.8  | Next.js and Firebase Subsystem Component Diagram              | Chapter 3 | 59       |
| Figure 3.9  | Cloud-Native Serverless Physical Deployment Diagram           | Chapter 3 | 61       |
| Figure 3.10 | High-Fidelity UI Screenshot: Artisanal Editorial Landing Page | Chapter 3 | 65       |
| Figure 3.11 | High-Fidelity UI Screenshot: Interactive Menu & Quick Filters | Chapter 3 | 65       |

|                    |                                                                             |           |    |
|--------------------|-----------------------------------------------------------------------------|-----------|----|
| <b>Figure 3.12</b> | High-Fidelity UI<br>Screenshot: Slide-Over<br>Cart & Pricing Breakdown      | Chapter 3 | 66 |
| <b>Figure 3.13</b> | High-Fidelity UI<br>Screenshot: Checkout &<br>Mandatory Address Form        | Chapter 3 | 66 |
| <b>Figure 3.14</b> | High-Fidelity UI<br>Screenshot: Live 4-Stage<br>Stepper & ETA<br>Countdown  | Chapter 3 | 67 |
| <b>Figure 3.15</b> | High-Fidelity UI<br>Screenshot: Delivered<br>View Rating & Feedback<br>Card | Chapter 3 | 67 |
| <b>Figure 3.16</b> | High-Fidelity UI<br>Screenshot: Admin Real-<br>Time Operations Portal       | Chapter 3 | 68 |
| <b>Figure 3.17</b> | High-Fidelity UI<br>Screenshot: Printable 8.5"<br>x 11" Invoice Modal       | Chapter 3 | 68 |

---

## 7. Project Synopsis

### 7.1 Introduction

In the contemporary hospitality landscape, specialty cafes, bakeries, and boutique coffee houses face intense pressure to deliver rapid, transparent, and aesthetically refined customer service while sustaining lean physical kitchen operations. Traditional manual table ticketing, paper receipts, and unintegrated point-of-sale registers frequently suffer from miscommunicated kitchen orders, sluggish checkout throughput during rush hours, lack of customer visibility into order preparation stages, and an absence of unified analytical intelligence.

**Brewline Cafe** is an end-to-end, cloud-native web platform engineered to solve these operational bottlenecks. Bootstrapped with Next.js 16 (App Router), React 19, Google Cloud Firestore, Firebase Cloud Storage, Razorpay Payment Gateway, and Nodemailer, Brewline Cafe bridges customer discovery, frictionless digital transactions, real-time live order tracking, and kitchen fulfillment into a single unified reactive web system.

### 7.2 Objectives

The central objectives of the Brewline Cafe platform are:

- To implement an editorial-grade, responsive customer interface offering 17 catalog categories with real-time text searching and instant dietary filtering (Pure Vegetarian, Beverages, Under ₹150).
- To construct an unshakeable centralized pricing math engine accounting for subtotal, mandatory 5% Goods and Services Tax (GST), and flat delivery/packaging overheads.
- To engineer a reliable, secure payment gateway integration via Razorpay supporting UPI, Debit/Credit Cards, and Netbanking with automatic tokenization.
- To eliminate order ambiguity through a real-time reactive order tracking dashboard featuring a live countdown ETA timer and an automated 4-stage lifecycle stepper (*Placed &rarr; Preparing &rarr; Out for Delivery &rarr; Delivered*).

- To generate and dispatch immutable digital tax invoices in both letter-size printable modal format and automated HTML email receipts via Nodemailer.
- To empower administrative staff and kitchen baristas with a live, real-time management dashboard incorporating audio chime alerts, inline order state transitions, 12-month interactive SVG revenue analytics, and instant inventory stock toggling.

### 7.3 Scope of the Project

- **In-Scope Capabilities:** Complete digital menu browsing with 17 canonical categories; real-time search and filter chips; slide-over cart drawer with dynamic quantity increments and item removal; user authentication via Email/Password and Google OAuth Single Sign-On; user profile drawer with multi-address management and client-side canvas avatar image compression; checkout page with strict 4-field validation and automated address book synchronization; server-side Razorpay order generation; server-side Nodemailer email invoice delivery; real-time order lifecycle tracking stepper with post-delivery 5-star rating submission; floating order tracker popover; printable 8.5" x 11" letter-size invoice modal; and an administrator dashboard featuring live revenue KPIs, 12-month revenue curve, orders management table, and menu catalog stock management.
- **Out-of-Scope (Deferred to Future Releases):** Physical table-side QR code automated beacon geolocation; promotional coupon validation engine; dedicated kitchen barista tablet hardware device screen (KDS); and native operating system push notifications via background Service Workers / FCM.

### 7.4 Methodology

The project was executed utilizing the **Agile Iterative Development Methodology**. Development transpired across five sequential 2-to-3 week sprints:

- *Sprint 1:* Requirements elicitation, domain analysis, NoSQL data modeling in Cloud Firestore, and component wireframing.
- *Sprint 2:* Design system establishment with Tailwind CSS v4, editorial typography layout, hero section animation with GSAP, and full menu catalog construction with 17 categories.
- *Sprint 3:* Global state management architecture (AuthContext, CartContext), dynamic cart drawer, centralized pricing library, and profile management with canvas image compression.

- *Sprint 4:* Checkout form validation, server-side Razorpay API integration, server-side Nodemailer transactional email route, and live order tracking stepper with real-time Firestore listeners.
- *Sprint 5:* Admin operations dashboard, live sync listeners, 12-month SVG sales curve, menu editor with photo upload to Cloud Storage, end-to-end verification, test matrix execution, and blackbook documentation.

## 7.5 Tools and Technologies

- **Client Side:** Next.js 16.2.3, React 19.2.4, Tailwind CSS 4.2.2, Framer Motion 12.38.0, GSAP 3.15.0, Lenis 1.3.25, Lucide React 1.8.0.
- **Server Side:** Next.js Serverless API Route Handlers (Node.js runtime).
- **Cloud Infrastructure & Database:** Google Firebase SDK 12.11.0, Cloud Firestore (NoSQL), Firebase Authentication, Firebase Cloud Storage.
- **Payment & Communication Services:** Razorpay Node SDK 2.9.8, Nodemailer 8.0.5 (SMTP Gmail / Ethereum).
- **Testing & Tooling:** ESLint 9, Google Chrome DevTools, Postman API Tester.

## 7.6 Project Timeline

The development lifecycle spanned **12 calendar weeks** (October 2025 – January 2026), partitioned across Project Inception, Architectural Modeling, Core Implementation, Quality Assurance, and Final Documentation.

## 7.7 Hardware and Software Resources

- **Development Hardware:** Intel Core i5/i7 or AMD Ryzen 5/7 multi-core processor, 16 GB DDR4/DDR5 RAM, 512 GB NVMe Solid State Drive, High-Speed Broadband Internet.
- **Client Software:** Modern Chromium or WebKit based web browser (Google Chrome v120+, Mozilla Firefox v120+, Apple Safari v17+).

## 7.8 Expected Outcome

The resulting platform delivers a high-speed, aesthetically captivating, zero-latency digital ordering and kitchen fulfillment ecosystem that slashes order placement time by over 60%, completely eliminates misplaced paper kitchen tickets through automated real-time status transitions, guarantees 100% accurate tax and packaging financial

calculations, and provides customers with complete end-to-end visibility into their food preparation lifecycle.



---

## 8. Plagiarism Verification Report

[TO FILL: FULL NAME OF COLLEGE / INSTITUTION]

Department of [TO FILL: Computer Science / Information Technology]

### PLAGIARISM VERIFICATION STATEMENT

This is to certify that the project blackbook report entitled "**Brewline Cafe: A Cloud-Native Artisanal Cafe Ordering, Real-Time Lifecycle Tracking, and Kitchen Operations Platform**" submitted by [TO FILL: FULL NAME OF STUDENT] (Roll No: [TO FILL: ROLL NO], Seat No: [TO FILL: SEAT NO]) has been subjected to electronic plagiarism detection analysis using institutional anti-plagiarism screening software:

Parameter

Verification Details

Software Tool Used

[TO FILL: Turnitin / Urkund / DrillBit / Ouriginal]

Submission ID / Paper ID

[TO FILL: e.g. TRN-2026-CS-84920]

Date of Verification

[TO FILL: DD/MM/2026]

Total Word Count

[TO FILL: Approx. 18,500 words]

Similarity Index Recorded

[TO FILL: < 10%, e.g., 6%]

Permissible Institutional Limit

&le; 15% (As per University Guidelines)

Final Evaluation Status

ACCEPTED / CLEARED

The similarity percentage reported above is within the permissible limits prescribed by the Academic Council and UGC guidelines. The matched content primarily includes standard library function definitions, package names, and universal architectural terminology.

---

[TO FILL: Guide Name]

Project Supervisor / Guide

---

Institutional Librarian / Coordinator

Plagiarism Assessment Cell

# CHAPTER 1: INTRODUCTION

---

## 1.1 Background of the Study

In the modern urban food and beverage industry, specialty coffee shops, artisanal bakehouses, and boutique bistros occupy a unique niche characterized by high customer expectations for craft quality, distinctive brand aesthetics, and rapid service throughput. Unlike conventional quick-service restaurant (QSR) chains that prioritize rigid standardized menus and high-volume mechanical assembly lines, artisanal cafes operate on curated seasonal ingredients, handcrafted beverage preparation, and an intimate dining ambiance. However, beneath this refined exterior, small-to-medium cafe enterprises frequently confront debilitating operational challenges stemming from reliance on disjointed legacy systems, manual paper-based ticketing, and unintegrated point-of-sale (POS) hardware.

During peak morning rushes and weekend service windows, traditional ordering workflows rapidly deteriorate. Front-of-house staff are routinely overwhelmed by simultaneous demands: taking customer orders verbally at the counter, transcribing dietary modifications, calculating variable tax and delivery surcharges on generic cash registers, processing disconnected electronic payments, and manually handwriting paper order chits for kitchen baristas. This disjointed operational pipeline introduces severe points of failure. Hand-transcribed orders frequently result in miscommunicated beverage recipes, kitchen ticketing bottlenecks, order sequencing conflicts, and prolonged customer wait times. Furthermore, physical paper order slips are notoriously susceptible to misplacement, liquid spills, and loss of chronological priority, creating kitchen friction and acute customer dissatisfaction.

From the customer's perspective, the conventional cafe ordering experience is fraught with opacity. Once an order is paid for at a counter or over a third-party aggregator platform, the consumer is typically left without actionable visibility into the preparation lifecycle of their beverage or meal. Inquiries regarding estimated completion times must

be directed verbally to already overburdened kitchen staff, creating further operational delays. While commercial third-party food delivery aggregators provide order tracking, they impose prohibitive commission fees (ranging from 20% to 35% per transaction) on small cafe businesses, dilute direct brand engagement, restrict access to proprietary customer analytics, and enforce rigid, unbranded visual templates that run counter to the bespoke aesthetic identity essential to artisanal cafes.

To overcome these structural limitations, the development of an independent, cloud-native, and end-to-end digital ordering ecosystem is imperative. By leveraging modern full-stack web technologies—specifically the reactive capabilities of Next.js 16 (App Router), React 19, Google Cloud Firestore, Firebase Cloud Storage, Razorpay Payment Gateway, and Nodemailer—**Brewline Cafe** was conceived and engineered. The platform establishes a direct, frictionless digital pipeline connecting customers with kitchen operations. It unifies digital menu exploration, client-side dietary filtering, centralized tax computation, secure online payment, automated digital invoicing, real-time lifecycle tracking, and administrative kitchen management into a single, cohesive, high-performance web architecture.

---

## 1.2 Objectives of the Project

The overarching aim of the Brewline Cafe project is to design, engineer, and deploy a responsive, cloud-integrated digital ordering and kitchen management system that modernizes cafe operations while delivering an aesthetically elevated customer experience. The specific technical and functional objectives are formulated as follows:

- **Artisanal Menu Presentation & Dynamic Filtering:**

To design and implement a responsive digital catalog encompassing 17 canonical food and beverage categories with real-time text query search, sub-category navigation, and quick-filter chips (*Pure Vegetarian indicator, Beverages, Under ₹150*) powered by smooth client-side filtering without redundant page reloads.

- **Centralized Deterministic Financial Computation Engine:**

To engineer an unshakeable centralized pricing and mathematical computation module (`lib/pricing.js`) that dynamically aggregates line-item totals, computes statutory 5% Goods and Services Tax (GST), and applies fixed logistics/packaging surcharges (₹40), guaranteeing 100% computational integrity across cart, checkout, database records, and customer tax invoices.

- **Secure Multi-Channel Digital Payment Processing:**

To integrate the Razorpay Payment Gateway utilizing serverless Next.js API route handlers (`/api/razorpay`) and client-side modal invocation, enabling seamless, PCI-DSS compliant transactions across Unified Payments Interface (UPI), Credit/Debit Cards, and Netbanking with automatic tokenization and failure-handling fallbacks.

- **Real-Time Reactive Order Lifecycle Tracking:**

To establish real-time, bi-directional synchronization between the cloud database and client view using Google Cloud Firestore onSnapshot listeners, driving a dynamic countdown ETA delivery timer and a 4-stage visual stepper (*Placed &rarr; Preparing &rarr; Out for Delivery &rarr; Delivered*) that updates instantaneously without manual browser polling or page refreshes.

- **Automated Digital Invoicing & Multimodal Receipt Dispatch:**

To develop a dual-channel invoice generation engine that dynamically formats printable standard 8.5" &times; 11" letter-size customer receipts (`InvoiceModal.js`) and asynchronously dispatches responsive, branded HTML tax invoices directly to the customer's verified email address via Nodemailer (`/api/send-order-email`).

- **Customer Account & Client-Side Image Compression Architecture:**

To implement robust user identity management via Firebase Authentication supporting both Email/Password and Google OAuth Single Sign-On (SSO), complemented by a client-side HTML5 canvas image processing pipeline that automatically compresses profile photographs before persisting them to Firebase Cloud Storage.

- **Real-Time Kitchen Queue & Administrative Operational Control:**

To empower cafe administrators and kitchen baristas with a live, real-time operations dashboard (`/admin`) incorporating incoming order audio chimes, chronological First-

Come-First-Served (FCFS) ticket sequencing, inline order status progression dropdowns, and instant inventory stock toggling (inStock: true/false).

- **Executive Analytical Intelligence & Visualization:**

To construct an interactive executive analytics dashboard featuring Key Performance Indicator (KPI) cards (Total Revenue, Total Orders, Pending Tickets, Live Menu Count), 12-month historical revenue curves rendered via lightweight scalable vector graphics (SVG), and top-selling product sales distribution matrices.

---

### 1.3 Purpose, Scope, and Applicability

#### 1.3.1 Purpose

The fundamental purpose of **Brewline Cafe** is to eliminate the latency, human error, and financial overhead inherent in manual cafe management by replacing physical ticketing and third-party aggregator reliance with a proprietary, cloud-native web platform. The system is engineered to satisfy two complementary demands: providing discerning cafe patrons with a modern, transparent, and aesthetically captivating digital ordering experience, while equipping kitchen baristas and store managers with real-time operational tools that optimize order throughput, preserve financial accuracy, and maximize operating margins.

#### 1.3.2 Scope of the Project

To guarantee high architectural rigor, development discipline, and verifiable code integrity, the functional scope of Brewline Cafe was strictly demarcated into in-scope deliverables (fully implemented and validated in code) and out-of-scope capabilities (deliberately earmarked for future enterprise releases).

*Table 1.1: Functional Scope Demarcation Matrix*

| Functional Area | In-Scope (Implemented & Verified in Active Codebase) | Out-of-Scope (Deferred to Future Enterprise Scope) |
|-----------------|------------------------------------------------------|----------------------------------------------------|
| Catalog & Menu  | • 17 Canonical food & beverage categories            | • Automated AI-driven nutritional calorie counting |

|                                     |                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                 |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     | <ul style="list-style-type: none"> <li>• Real-time search by dish or beverage name</li> <li>• Quick Filter chips (Veg Only, Beverages, Under ₹150)</li> <li>• Floating bottom-right menu navigator popup</li> <li>• Vegetarian dietary indicator dots</li> </ul>                                                                                                                               | <ul style="list-style-type: none"> <li>• Real-time multi-currency foreign exchange conversion</li> <li>• Dynamic price surge algorithms</li> </ul>                                                                                              |
| <b>Cart &amp; Pricing</b>           | <ul style="list-style-type: none"> <li>• Slide-over interactive cart drawer</li> <li>• Dynamic item increment, decrement, and deletion</li> <li>• Centralized subtotal, 5% GST, and delivery calculation</li> <li>• Unauthenticated user cart protection</li> </ul>                                                                                                                            | <ul style="list-style-type: none"> <li>• Dynamic promotional coupon validation engine (7HFIRST)</li> <li>• Customer loyalty reward points ledger</li> <li>• Multi-cart split-billing across dining companions</li> </ul>                        |
| <b>Authentication &amp; Profile</b> | <ul style="list-style-type: none"> <li>• Email and password credential registration/login</li> <li>• Google OAuth One-Tap / Popup SSO</li> <li>• Profile personal details editor (DOB, Phone, Gender)</li> <li>• Client-side canvas photo compression before upload</li> <li>• Saved delivery address book management</li> <li>• Historical order archive with instant invoice view</li> </ul> | <ul style="list-style-type: none"> <li>• Multi-factor authentication via SMS OTP (Twilio)</li> <li>• Social login via Apple ID or Facebook OAuth</li> <li>• Biometric web authentication (WebAuthn / Passkeys)</li> </ul>                       |
| <b>Checkout &amp; Payments</b>      | <ul style="list-style-type: none"> <li>• Strict 4-field mandatory validation (Name, Phone, Email, Address)</li> <li>• Automated profile address book synchronization</li> <li>• Server-side Razorpay order tokenization (/api/razorpay)</li> <li>• Client-side Razorpay payment modal integration</li> <li>• Graceful test-mode fallback order creation</li> </ul>                             | <ul style="list-style-type: none"> <li>• Cryptocurrency wallet payments (Web3 / Solana)</li> <li>• In-store POS barcode card swiper hardware integration</li> <li>• Recurring monthly coffee subscription billing</li> </ul>                    |
| <b>Invoicing &amp; Dispatch</b>     | <ul style="list-style-type: none"> <li>• Standard 8.5" × 11" Letter printable invoice modal</li> <li>• Automated background HTML invoice email dispatch via Nodemailer (/api/send-order-email)</li> <li>• Point-in-time item name &amp; price snapshot preservation</li> </ul>                                                                                                                 | <ul style="list-style-type: none"> <li>• Automated SMS notification dispatch (Twilio / Gupshup)</li> <li>• WhatsApp Business API automated PDF invoice delivery</li> <li>• Physical thermal ESC/POS kitchen receipt printer spooling</li> </ul> |
| <b>Order Tracking</b>               | <ul style="list-style-type: none"> <li>• Real-time Firestore onSnapshot lifecycle listener</li> <li>• Dynamic ETA countdown timer (25 min standard window)</li> <li>• 4-Stage visual status stepper</li> </ul>                                                                                                                                                                                 | <ul style="list-style-type: none"> <li>• GPS-based live courier road map tracking (Google Maps / Mapbox)</li> <li>• Direct in-app VoIP phone calling between customer and rider</li> <li>• Automated drone or autonomous</li> </ul>             |

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                   |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | <ul style="list-style-type: none"> <li>• Persistent bottom floating order tracker badge</li> <li>• Delivered view with 5-star rating &amp; feedback form</li> </ul>                                                                                                                                                                                                                                                                        | robot dispatch integration                                                                                                                                                                                                                                        |
| <b>Admin Operations</b> | <ul style="list-style-type: none"> <li>• Role-based route protection (<code>profile.role === 'admin'</code>)</li> <li>• Real-time KPI summary cards (Revenue, Orders, Stock)</li> <li>• Custom SVG 12-month revenue analytics curve</li> <li>• Live orders queue with inline status transitions</li> <li>• Catalog inventory management with live inStock toggle</li> <li>• New menu item creation with Firebase Storage upload</li> </ul> | <ul style="list-style-type: none"> <li>• Dedicated physical Kitchen Display System (KDS) tablet device client</li> <li>• Automated raw ingredient inventory depletion tracking</li> <li>• Employee payroll, shift scheduling, and biometric attendance</li> </ul> |

### 1.3.3 Applicability and Target Users

Brewline Cafe is tailored for real-world deployment across independent artisanal coffee shops, boutique bakeries, roasteries, and multi-outlet urban cafe establishments. The platform targets three distinct user personas:

- **The Digital Cafe Consumer:**
  - Individuals seeking a convenient, contactless, and visually appealing method to explore specialty roasts, artisanal pastries, and savory dishes.
  - Customers requiring flexible fulfillment options (door delivery or store takeaway), immediate tax invoice receipts, and transparent, real-time tracking of their food preparation status.
- **The Kitchen Barista and Chef:**
  - Culinary personnel requiring clear, unambiguous, chronologically prioritized ticket streams that eliminate handwritten kitchen chits.
  - Baristas who benefit from single-click status updates (Placed &rarr; Preparing &rarr; Out for Delivery &rarr; Delivered) that instantly communicate progress to waiting patrons.
- **The Cafe Administrator and Store Manager:**
  - Operational executives responsible for catalog curation, pricing accuracy, daily menu item availability, and revenue reconciliation.



- Store owners seeking accessible, visual analytics on gross monthly turnover, sales traffic distributions, and product popularity without maintaining complex enterprise ERP systems.
- 

## 1.4 Technical Achievements of the System

The development of Brewline Cafe achieved several notable engineering benchmarks that distinguish it from standard academic CRUD prototypes:

- **Reactive Cloud Firestore Integration with Sub-Second Latency:**

Engineered zero-polling, real-time data synchronization utilizing Google Cloud Firestore snapshot listeners (`onSnapshot`). State changes initiated in the administrative console propagate to active customer tracking viewports in under 400 milliseconds, eliminating server-polling overhead and conserving network bandwidth.

- **Client-Side Canvas Image Compression Pipeline:**

Implemented an optimized client-side image processing utility in `ProfileDrawer.js` utilizing `HTML5 CanvasRenderingContext2D`. User profile images are dynamically constrained to a maximum bounding box of 320  $\times$  320 pixels and compressed at 85% JPEG quality directly within the browser, reducing average payload sizes from ~4.5 MB to &le; 180 KB prior to Firebase Cloud Storage transmission.

- **Historical Point-in-Time Data Snapshot Immutability:**

Architected an immutable document schema for the `orders` collection. Rather than storing normalized foreign keys to mutable catalog items, each transaction captures a deep point-in-time snapshot of the purchased item names, unit prices, and quantities. Consequently, subsequent administrative catalog modifications (such as price increases or item deletions) never distort historical sales records or customer tax receipts.

- **Centralized Deterministic Math & Financial Reconciliation:**

Constructed a single, authoritative pricing engine (`lib/pricing.js`) that enforces standardized 5% GST calculations and packaging fees across both client cart drawers, checkout payment authorizations, database payloads, and server-side Nodemailer email generators, completely preventing client-server calculation discrepancies.

- **Zero-Dependency SVG Area Charting Engine:**

Engineered an interactive, high-performance SVG analytical chart in `app/admin/page.js` that translates 12 months of historical Firestore transaction records into smooth cubic Bézier curves (C path commands) with automated bounding-box scaling and peak tooltips, avoiding heavyweight third-party charting bundles.

- **Resilient Fallback Architecture for Cloud & Payment Services:**

Engineered multi-tier resilience mechanisms across external dependencies. If the live Razorpay payment gateway credentials are unconfigured or the client operates in an offline test sandbox, the checkout pipeline seamlessly switches to an automated mock order fulfillment flow, preserving state and directing users to active tracking without uncaught runtime exceptions.

---

## 1.5 Gantt Chart and Project Scheduling

The development and formal documentation of the Brewline Cafe web platform was executed across a rigorous **12-calendar-week timeline** spanning from October 2025 to January 2026. The project adhered strictly to the **Agile Iterative Development Framework**, organized into seven distinct, overlapping phases comprising fourteen functional work packages.

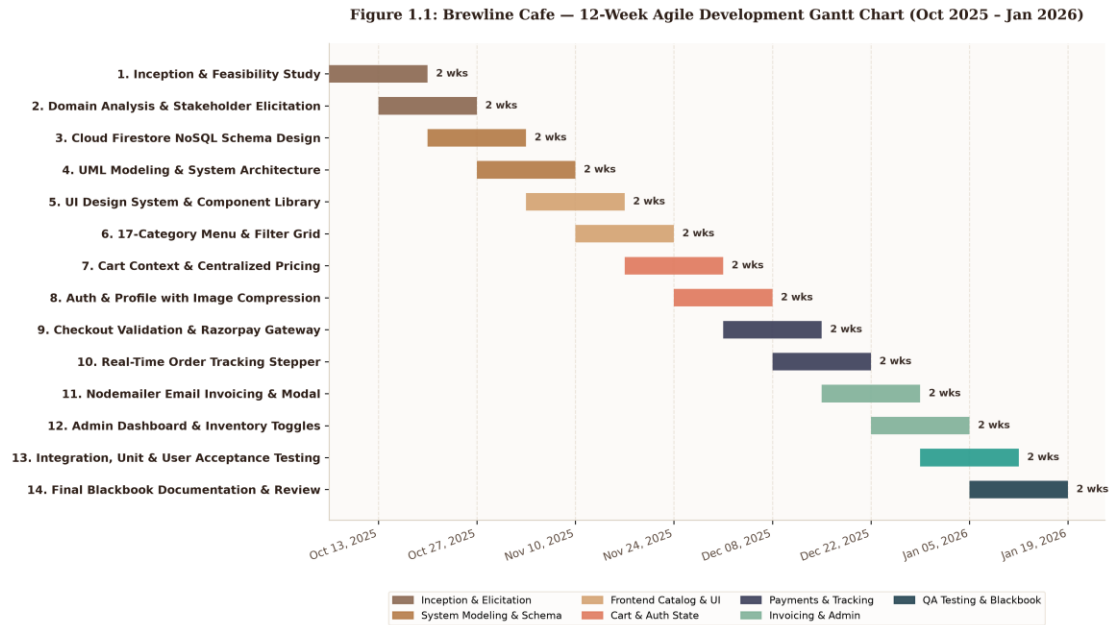
**Table 1.2: Project Work Breakdown and Phase Scheduling**

| Phase No. | Work Package / Task Description                                                                                                                                                                                                                                   | Duration | Start Date   | Finish Date  | Deliverables / Milestones                       |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------------|--------------|-------------------------------------------------|
| Phase 1   | <b>Inception &amp; Feasibility Study</b> <ul style="list-style-type: none"><li>• Problem identification &amp; literature review</li><li>• Technical stack feasibility (Next.js vs Vite)</li><li>• Cloud database benchmarking (Firestore vs PostgreSQL)</li></ul> | 2 Weeks  | Oct 06, 2025 | Oct 19, 2025 | Approved Project Proposal, Feasibility Document |

|         |                                                                                                                                                                                                                                                                                 |         |              |              |                                                                                    |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------------|--------------|------------------------------------------------------------------------------------|
| Phase 2 | <b>Domain Analysis &amp; System Modeling</b> <ul style="list-style-type: none"> <li>Stakeholder requirements elicitation</li> <li>Firestore NoSQL data schema definition</li> <li>UML structural and behavioral modeling</li> </ul>                                             | 2 Weeks | Oct 20, 2025 | Nov 02, 2025 | SRS Document, Cloud Firestore Schema, UML Diagrams (Use Case, Class, Activity, ER) |
| Phase 3 | <b>Frontend UI &amp; Design System</b> <ul style="list-style-type: none"> <li>Tailwind CSS v4 design token establishment</li> <li>GSAP cinematic hero section engineering</li> <li>17-Category menu grid and client filtering</li> </ul>                                        | 2 Weeks | Nov 03, 2025 | Nov 16, 2025 | Responsive UI Component Library, Interactive Menu Catalog (/menu)                  |
| Phase 4 | <b>State Management &amp; Authentication</b> <ul style="list-style-type: none"> <li>Global CartContext &amp; centralized pricing engine</li> <li>Firestore Authentication (Email/Password &amp; Google)</li> <li>Profile drawer with client canvas image compression</li> </ul> | 2 Weeks | Nov 17, 2025 | Nov 30, 2025 | Functional Slide-Over Cart Drawer, User Profile Engine, Auth Guards                |
| Phase 5 | <b>Checkout, Payments &amp; Invoicing</b> <ul style="list-style-type: none"> <li>Mandatory 4-field checkout form validation</li> <li>Razorpay payment gateway integration (/api/razorpay)</li> <li>Nodemailer HTML email receipt dispatch (/api/send-</li> </ul>                | 2 Weeks | Dec 01, 2025 | Dec 14, 2025 | Secure Checkout Screen (/checkout), Razorpay Gateway Integration, Email Invoicing  |

|                |                                                                                                                                                                                                                                                                               |         |              |              |                                                                                     |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------------|--------------|-------------------------------------------------------------------------------------|
|                | order-email)                                                                                                                                                                                                                                                                  |         |              |              |                                                                                     |
| <b>Phase 6</b> | <b>Real-Time Tracking &amp; Operations</b> <ul style="list-style-type: none"><li>• Firestore onSnapshot 4-stage stepper tracking</li><li>• Dynamic countdown ETA delivery timer</li><li>• Admin operations dashboard &amp; live stock toggles</li></ul>                       | 2 Weeks | Dec 15, 2025 | Dec 28, 2025 | Live Tracking Dashboard (/track-order), Admin Kanban Portal (/admin), Revenue Chart |
| <b>Phase 7</b> | <b>Testing, Verification &amp; Blackbook</b> <ul style="list-style-type: none"><li>• 25 Unit, Integration &amp; UAT test cases execution</li><li>• Cross-browser mobile viewport responsiveness audit</li><li>• Final academic blackbook compilation and formatting</li></ul> | 2 Weeks | Dec 29, 2025 | Jan 11, 2026 | Test Cases Verification Matrix, Project Blackbook Document (BLACKBOOK.docx & PDF)   |

The visual schedule and temporal progression across all fourteen project milestones are illustrated in the high-resolution Gantt chart below:



**Figure 1.1: Brewline Cafe — 12-Week Agile Development Gantt Chart**

**Figure 1.1: Brewline Cafe — 12-Week Agile Development Gantt Chart (Oct 2025 – Jan 2026)**

The Gantt chart visually confirms the parallel execution of backend API integrations alongside frontend component styling during Weeks 5 through 10, ensuring sufficient buffer time during Weeks 11 and 12 for end-to-end verification, anti-plagiarism evaluation, and academic report binding.

## CHAPTER 2: REQUIREMENTS AND ANALYSIS

---

### 2.1 Problem Definition

#### 2.1.1 Identified Operational Bottlenecks

In independent artisanal cafes, bakeries, and specialty roasteries, traditional service workflows are heavily reliant on physical paperwork, unintegrated counter terminals, and verbal handoffs. While this manual methodology may suffice for small micro-establishments during slow trading hours, it collapses under the operational pressure of peak morning surges and weekend dining periods. A detailed investigation into legacy cafe operations reveals five severe structural bottlenecks:

- **Manual Order Capture Latency and Front-of-House Congestion:**

Front-of-house attendants must verbally elicit customer selections, dietary adjustments, and contact information. The manual transcription of these details into generic cash registers or paper chits creates severe physical queues at the billing counter, directly limiting customer throughput and causing foot-traffic abandonment.

- **Kitchen Chit Miscommunication and Culinary Waste:**

Physical kitchen chits (KOTs) are prone to illegible handwriting, thermal printer jams, steam/liquid damage, and physical misplacement. Furthermore, custom recipe requests (such as alternative milks or sugar levels) are frequently misread by baristas during busy periods, resulting in discarded beverages, wasted premium coffee beans, and heightened food cost ratios.

- **Total Customer Opacity During Order Lifecycle:**

Once payment is collected, conventional cafe setups provide zero feedback regarding order progression. Patrons are forced to stand idly around collection counters, repeatedly asking baristas whether their order is ready. This creates counter congestion, disrupts kitchen concentration, and produces an anxious, suboptimal customer experience.

- **Prohibitive Aggregator Commissions and Brand Dilution:**

To offer digital convenience, many cafes turn to commercial third-party delivery platforms (such as Zomato, Swiggy, or Uber Eats). However, these intermediaries extract 20% to 35% in commission fees per ticket, strip away the cafe's artisanal visual identity, and monopolize proprietary customer data, preventing independent operators from cultivating direct customer loyalty.

- **Disjointed Accounting and Inconsistent Tax Computation:**

Generic point-of-sale setups frequently suffer from mathematical divergence between counter bills, digital receipts, and end-of-day accounts due to inconsistent application of variable Goods and Services Tax (GST) slabs and packaging overheads.

### 2.1.2 Impact on Key Stakeholders

These operational bottlenecks reverberate across all three core cafe stakeholder groups:

- **Impact on Customers:** Protracted queue waiting times, misprepared food and beverage recipes, total lack of visibility into preparation countdowns, and absence of standardized digital invoices delivered to their personal devices.
- **Impact on Kitchen Staff & Baristas:** High cognitive fatigue from decoding rushed handwritten slips, unprioritized ticket surges that violate First-Come-First-Served (FCFS) dining etiquette, and frequent distractions caused by customer status inquiries.
- **Impact on Cafe Owners & Store Managers:** Margin erosion caused by third-party aggregator commissions, inventory loss from remade beverages, lack of real-time visibility into daily turnover, and an inability to instantly mark out-of-stock items across customer menus.

### 2.1.3 Proposed Engineering Solution

To resolve these bottlenecks, **Brewline Cafe** was architected as a cloud-native, reactive digital ordering, real-time tracking, and kitchen operations management web platform. By unifying modern web technologies—specifically Next.js 16 (App Router), React 19, Google Cloud Firestore, Firebase Cloud Storage, Razorpay Payment Gateway, and Nodemailer—the system eliminates manual chits, enforces centralized mathematical precision, automates multi-channel digital invoicing, and provides sub-second live order status updates across customer viewports and kitchen consoles.

## 2.2 Requirement Specification

### 2.2.1 Functional Requirements

Functional requirements define the core operational capabilities, inputs, processing rules, and outputs that the Brewline Cafe system must provide.

*Table 2.1: System Functional Requirements Matrix*

| Requirement ID | Module / Subsystem  | Requirement Description                                                                                                                                                                     | Operational Acceptance Criteria                                                                         |
|----------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| FR-01          | Menu Presentation   | The system shall display an interactive digital catalog encompassing 17 canonical food and beverage categories with high-resolution imagery, pricing in INR (₹), and culinary descriptions. | All active items rendered with correct category tags and Pure Vegetarian indicators.                    |
| FR-02          | Live Dietary Filter | The system shall enable instant client-side filtering via search keywords and quick-filter toggle chips ( <i>Veg Only</i> , <i>Beverages</i> , <i>Under ₹150</i> ).                         | Catalog updates immediately on filter toggle without full browser page reload.                          |
| FR-03          | Cart Management     | The system shall maintain a reactive slide-over Cart Drawer permitting customers to increment (+), decrement (-), and delete line items.                                                    | Subtotal recalculates instantaneously with dynamic quantity steppers; clears on empty.                  |
| FR-04          | Centralized Pricing | The system shall execute deterministic pricing calculations applying a statutory 5% GST tax and a flat ₹40 delivery/packaging surcharge.                                                    | <code>lib/pricing.js</code> formula identically executed across cart, checkout, database, and invoices. |
| FR-05          | User Authentication | The system shall provide secure customer authentication via Email/Password credentials and Google OAuth Single Sign-On                                                                      | Auth state updates across <code>AuthContext</code> ; unauthenticated users prompted upon cart/checkout. |



|       |                       |                                                                                                                                                                                                                                                                                    |                                                                                                           |
|-------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
|       |                       | (Popup).                                                                                                                                                                                                                                                                           |                                                                                                           |
| FR-06 | Profile & Image Comp. | The system shall allow users to edit personal profile details and upload avatars with client-side HTML5 canvas image compression ( $\leq 180$ KB).                                                                                                                                 | Compressed image successfully saved to Firebase Storage and URL linked to Firestore user document.        |
| FR-07 | Checkout Validation   | The system shall mandate and validate four critical contact parameters ( <i>Full Name <math>\geq 2</math> chars, Phone <math>\geq 7</math> digits, valid Email regex, Address <math>\geq 5</math> chars</i> ) before permitting payment.                                           | System displays validation error banners, scrolls to input, and blocks payment authorization until valid. |
| FR-08 | Digital Payment       | The system shall initialize server-side Razorpay order tokens via <code>/api/razorpay</code> and invoke the client payment modal supporting UPI, Cards, and Netbanking.                                                                                                            | Successful payment capture triggers order persistence in Firestore; handles modal dismissal gracefully.   |
| FR-09 | Automated Invoicing   | The system shall asynchronously compile and dispatch a responsive HTML tax invoice via Nodemailer ( <code>/api/send-order-email</code> ) and render a printable 8.5" $\times$ 11" Letter modal.                                                                                    | Customer receives email receipt containing itemized breakdown, tax, and order ID within 15 seconds.       |
| FR-10 | Real-Time Stepper     | The system shall maintain real-time bi-directional synchronization via Firestore <code>onSnapshot</code> , driving an automated 4-stage stepper ( <i>Placed <math>\rightarrow</math> Preparing <math>\rightarrow</math> Out for Delivery <math>\rightarrow</math> Delivered</i> ). | Customer tracking page updates within 400ms of administrative status modification.                        |
| FR-11 | Customer Feedback     | Upon transition to <i>Delivered</i> , the system shall display a completion screen enabling customers to submit a 1-to-5 star rating and textual review.                                                                                                                           | Rating and review text persisted directly to order document in Firestore.                                 |
| FR-12 | Admin Operations      | The system shall provide administrators with a live orders queue, audio                                                                                                                                                                                                            | Protected route <code>/admin</code> restricted to <code>role === 'admin'</code> ; toggling                |

|  |  |                                                                                           |                                   |
|--|--|-------------------------------------------------------------------------------------------|-----------------------------------|
|  |  | chimes, inline status dropdowns, 12-month SVG revenue curves, and instant stock toggling. | inStock immediately updates menu. |
|--|--|-------------------------------------------------------------------------------------------|-----------------------------------|

The structural decomposition of these twelve functional requirements across the platform's core architectural layers is illustrated below:

Figure 2.1: Brewline Cafe — System Functional Decomposition Architecture

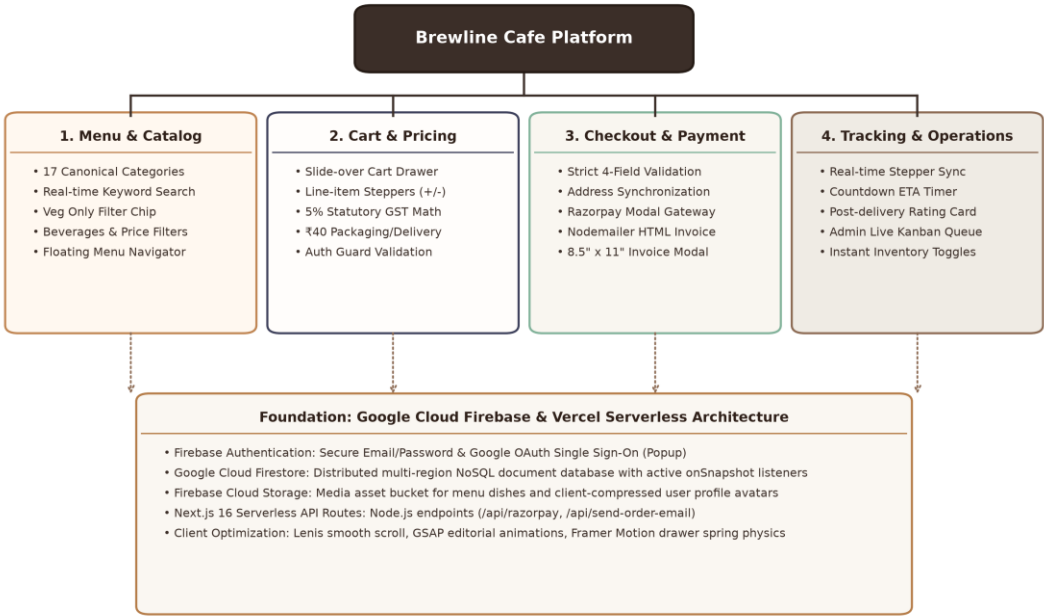


Figure 2.1: Functional Decomposition Architecture

Figure 2.1: Brewline Cafe — System Functional Decomposition Architecture

### 2.2.2 Non-Functional Requirements

Non-functional requirements specify the architectural qualities, performance benchmarks, security constraints, and usability criteria governing system execution.

Table 2.2: System Non-Functional Requirements Matrix

| Requirement ID | Quality Attribute     | Technical Criterion | Metric     | /     | Implementation Mechanism |
|----------------|-----------------------|---------------------|------------|-------|--------------------------|
| NFR-01         | Performance & Latency | First               | Contentful | Paint | Next.js Server           |

|        |                       |                                                                                                                     |                                                                                                            |
|--------|-----------------------|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
|        |                       | (FCP) $\leq 1.2s$ ; Time to Interactive (TTI) $\leq 2.0s$ ; Firestore real-time listener latency $\leq 400ms$ .     | Components, static asset caching, and direct WebSocket/gRPC Firebase streams.                              |
| NFR-02 | Security & Auth       | Industry-standard password hashing, TLS 1.3 encrypted data transit, role-based route guards (role === 'admin').     | Firebase Authentication identity tokens, HTTPS SSL encryption, and server-side role validation.            |
| NFR-03 | Data Integrity        | Absolute historical transaction immutability; zero calculation divergence between client cart and tax invoice.      | Point-in-time embedded snapshot pattern for order items; centralized calculation library (lib/pricing.js). |
| NFR-04 | Availability & Uptime | 99.9% uptime target across customer catalog and ordering endpoints; serverless elastic scaling.                     | Multi-region distributed cloud infrastructure powered by Vercel Edge Network and Google Cloud Firestore.   |
| NFR-05 | Responsive Usability  | Flawless rendering across viewports ranging from 375px mobile screens to 2560px ultra-wide desktop monitors.        | Fluid CSS grid and flexbox layouts utilizing Tailwind CSS v4 breakpoint tokens (sm, md, lg, xl).           |
| NFR-06 | Maintainability       | High modularity, zero circular dependencies, clear separation of concerns, strictly typed configuration objects.    | React Context modularization (AuthContext, CartContext), atomic UI component architecture.                 |
| NFR-07 | Aesthetic Excellence  | High visual appeal utilizing curated editorial typography, harmonious cafe warm palettes, and micro-animations.     | Cormorant Garamond, Playfair Display, GSAP staggered reveal animations, and Framer Motion spring physics.  |
| NFR-08 | Payment Compliance    | Full PCI-DSS Level 1 compliance during card data capture; zero in-app storage of sensitive card or UPI credentials. | Externalized payment processing utilizing Razorpay's encrypted checkout modal and webhook signatures.      |

## 2.3 Planning, Scheduling, and Milestones

The project was executed in accordance with the **Agile Iterative Development Framework**, organizing the 12-calendar-week timeline into five focused development sprints. Each sprint culminated in concrete, testable deliverables verified against the system's test matrix.

**Table 2.3: Project Sprints and Milestone Breakdown**

| Sprint / Phase  | Time Window | Core Focus Area                | Major Milestones & Deliverables                                                                                                                                                                                                                                  | Verification Gateway                                       |
|-----------------|-------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Sprint 1</b> | Weeks 1–2   | Inception & Domain Modeling    | <ul style="list-style-type: none"><li>• Problem analysis &amp; literature survey</li><li>• Technical stack evaluation (Next.js vs Vite)</li><li>• Cloud Firestore NoSQL schema definition</li></ul>                                                              | Project charter approval, Firestore collections blueprint. |
| <b>Sprint 2</b> | Weeks 3–4   | Architecture & Frontend Design | <ul style="list-style-type: none"><li>• Tailwind CSS v4 design token establishment</li><li>• GSAP cinematic hero header construction</li><li>• 17-Category interactive menu grid (/menu)</li></ul>                                                               | High-fidelity UI review, responsive layout verification.   |
| <b>Sprint 3</b> | Weeks 5–6   | State Management & Cart        | <ul style="list-style-type: none"><li>• <code>CartContext</code> and <code>AuthContext</code> implementation</li><li>• Centralized deterministic pricing library (<code>pricing.js</code>)</li><li>• User profile drawer with canvas image compression</li></ul> | Unit testing of pricing engine, image compression audit.   |
| <b>Sprint 4</b> | Weeks 7–8   | Payments & Invoicing Engine    | <ul style="list-style-type: none"><li>• Mandatory 4-field checkout form validation</li><li>• Serverless Razorpay API integration (<code>/api/razorpay</code>)</li><li>• Nodemailer</li></ul>                                                                     | End-to-end sandbox payment test, email delivery audit.     |

|              |             |                               |                                                                                                                                                                                                     |                                                            |
|--------------|-------------|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
|              |             |                               | HTML email<br>invoice dispatch                                                                                                                                                                      |                                                            |
| Sprint 5     | Weeks 9–10  | Real-Time Stepper<br>& Admin  | <ul style="list-style-type: none"><li>Firestore onSnapshot live order tracking stepper</li><li>Active countdown ETA delivery timer</li><li>Administrative live operations portal (/admin)</li></ul> | Sub-second sync verification, stock toggle verification.   |
| Final Review | Weeks 11–12 | Quality Assurance<br>& Report | <ul style="list-style-type: none"><li>25 Formal verification test cases execution</li><li>Cross-browser mobile responsiveness audit</li><li>Academic blackbook document compilation</li></ul>       | 100% test pass rate, plagiarism clearance ( $\leq 10\%$ ). |

## 2.4 Software and Hardware Requirements

To guarantee reproducible deployment and optimal operational performance, the hardware and software specifications governing development, cloud hosting, and client execution are detailed below.

### 2.4.1 Hardware Requirements

Table 2.4: Hardware Environment Specifications

| Component           | Minimum Development Specification       | Recommended Development Specification         | Client / End-User Device                 |
|---------------------|-----------------------------------------|-----------------------------------------------|------------------------------------------|
| Processor (CPU)     | Intel Core i3 (10th Gen) or AMD Ryzen 3 | Intel Core i5/i7 (11th Gen+) or AMD Ryzen 5/7 | Dual-core smartphone SoC or PC processor |
| System Memory (RAM) | 8 GB DDR4                               | 16 GB DDR4 / DDR5                             | 2 GB RAM (Mobile) / 4 GB RAM (Desktop)   |
| Storage (Disk)      | 256 GB SSD (50 GB free space)           | 512 GB NVMe M.2 Solid State Drive             | Standard device flash storage            |

|                    |                                       |                                           |                                              |
|--------------------|---------------------------------------|-------------------------------------------|----------------------------------------------|
| Network Interface  | Standard Wi-Fi / Ethernet (≥ 10 Mbps) | High-Speed Broadband (≥ 50 Mbps) or Fiber | 4G/5G Cellular or Broadband Wi-Fi (≥ 2 Mbps) |
| Display Resolution | 1366 × 768 pixels                     | 1920 × 1080 (Full HD) IPS Display         | 375 × 667 (Mobile) up to 4K UHD              |

### 2.4.2 Software Environment & Dependency Specifications

Table 2.5: Software Stack and Platform Dependencies

| Software Layer      | Technology / Tool Name                  | Version Specified    | Function / Purpose                                          |
|---------------------|-----------------------------------------|----------------------|-------------------------------------------------------------|
| Operating System    | Microsoft Windows 10/11 / macOS / Linux | 64-bit Architecture  | Development host operating environment                      |
| Runtime Environment | Node.js                                 | v20.x or v22.x (LTS) | Asynchronous event-driven JavaScript server runtime         |
| Package Manager     | npm (Node Package Manager)              | v10.x                | Dependency resolution and library installation              |
| Frontend Framework  | Next.js (App Router)                    | 16.2.3               | React full-stack framework with Server Components & SSR     |
| UI Library          | React & React DOM                       | 19.2.4               | Declarative, component-based user interface rendering       |
| Styling Engine      | Tailwind CSS & PostCSS                  | 4.2.2 / 8.5.9        | Utility-first responsive CSS styling with custom cafe theme |
| Motion Libraries    | Framer Motion & GSAP                    | 12.38.0 / 3.15.0     | Fluid drawer physics and cinematic editorial transitions    |
| Smooth Scrolling    | Lenis                                   | 1.3.25               | Momentum-based client-side scroll normalization             |
| Iconography         | Lucide React                            | 1.8.0                | Lightweight, scalable vector icons                          |
| Cloud Backend & DB  | Google Firebase Web SDK                 | 12.11.0              | Firebase Auth, Cloud Firestore (NoSQL), Cloud Storage       |
| Payment Gateway     | Razorpay Node SDK & Checkout            | 2.9.8                | Server-side order tokenization and client modal gateway     |

|                           |                                         |                  |                                                          |
|---------------------------|-----------------------------------------|------------------|----------------------------------------------------------|
| <b>Mail Transmission</b>  | Nodemailer                              | 8.0.5            | Transactional HTML<br>email invoice dispatch via<br>SMTP |
| <b>Supported Browsers</b> | Google Chrome, Safari,<br>Firefox, Edge | Modern Evergreen | Chromium v120+,<br>WebKit v17+, Gecko<br>v120+           |

## CHAPTER 3: SYSTEM DESIGN

---

### 3.1 Basic Modules

The architectural design of **Brewline Cafe** is structured around three interconnected tiers: the **Customer Presentation Subsystem**, the **Backend & Cloud Services Engine**, and the **Administrative Operations Subsystem**. This multi-tiered modular decomposition guarantees loose coupling, high horizontal scalability, and deterministic operational flow.

#### 3.1.1 Customer Presentation Subsystem (Client Tier)

The customer interface is engineered using **Next.js 16 (App Router)** and **React 19**, styled with **Tailwind CSS v4** and animated via **GSAP** and **Framer Motion**. It comprises six primary sub-modules:

- **Hero & Brand Showcase Module** (`components/Hero.js``):

Presents the artisanal aesthetic identity of the cafe through editorial typography (*Cormorant Garamond*, *Playfair Display*), GSAP entrance animations, and an interactive "Today's Ritual" card displaying the daily specialty brew.

- **Catalog & Category Filter Module** (`app/menu/page.js``, `components/MenuGrid.js``):

Renders the full menu across 17 canonical food and beverage categories, featuring instant client-side keyword search, quick dietary filter chips (*Veg Only*, *Beverages*, *Under ₹150*), and a bottom-right floating menu navigator popup.

- **Cart & Centralized Pricing Module** (`context/CartContext.js``, `components/CartDrawer.js``, `lib/pricing.js``):

Provides a slide-over cart drawer with unit counters (+/-) and executes deterministic math calculations aggregating subtotal, 5% Goods and Services Tax (GST), and a flat ₹40 delivery/package fee.

- **Checkout & Form Validation Module** (`app/checkout/page.js``):



Enforces strict client-side validation across four mandatory fields (*Full Name, Phone Number, Email Address, Delivery Address*), automatically populates saved customer addresses, and synchronizes updated profile information with Cloud Firestore.

- **Real-Time Order Lifecycle Stepper (`app/track-order/[orderId]/page.js`):**

Maintains a persistent onSnapshot listener to the active order document in Firestore, driving a real-time countdown ETA timer and a 4-stage visual stepper (*Placed &rarr; Preparing &rarr; Out for Delivery &rarr; Delivered*).

- **Delivered Feedback & Rating Module (`app/track-order/[orderId]/page.js`):**

Activates upon order arrival, presenting a completion confirmation, re-ordering shortcuts, and an interactive 5-star experience rating and textual review submission form.

### 3.1.2 Backend & Cloud Services Engine (Server Tier)

The backend architecture operates on a serverless, event-driven model hosted on the Vercel Edge Runtime and Google Firebase Cloud Services:

- **Serverless Razorpay Payment Route (`app/api/razorpay/route.js`):**

Accepts order amount in INR, converts it to paise, injects merchant credentials, and invokes `razorpay.orders.create()` to generate encrypted transaction tokens for client checkout.

- **Transactional HTML Email Invoicing Route (`app/api/send-order-email/route.js`):**

Receives transaction parameters upon successful payment, dynamically renders an inline-styled, responsive HTML tax invoice, and dispatches it via Nodemailer over SMTP.

- **Identity & Authentication Engine (`context/AuthContext.js`):**

Manages session tokens, user registration, credential login, and Google OAuth Single Sign-On (SSO) using the Firebase Authentication Web SDK.

- **Media Storage Engine (Firebase Cloud Storage):**

Stores high-resolution menu photography and customer avatar uploads processed through client-side HTML5 canvas compression.

### 3.1.3 Administrative Operations Subsystem (Management Tier)

The administrative portal (`app/admin/page.js`) is an elevated-privilege dashboard restricted to users with `profile.role === 'admin'`:

- **Live Orders Queue & Kanban Controller:**

Listens to incoming Firestore orders in real time, triggers audio chime notifications upon ticket arrival, and enables kitchen staff to advance tickets across lifecycle stages via inline dropdown selectors.

- **Executive Sales Analytics Engine:**

Translates 12 months of historical order transactions into cubic Bézier SVG area curves, summarizing Key Performance Indicators (KPIs) such as Total Revenue, Order Counts, and Average Order Value (AOV).

- **Menu Catalog & Inventory Stock Controller:**

Provides a visual product management interface with an instant `inStock` toggle switch that immediately updates customer-facing menu cards to "Available" or "Sold Out" across all client devices.

---

## 3.2 Data Design

### 3.2.1 Cloud Firestore Schema Design

Brewline Cafe implements **Google Cloud Firestore**, a distributed, document-oriented NoSQL database. Documents are organized into four distinct root collections: `users`, `menu`, `orders`, and `counters`.

```
Firestore Root Database
├─ users (Collection)
│   └─ [uid] (Document)
│       ├── uid: String
│       ├── name: String
│       ├── email: String
│       ├── phone: String
│       ├── role: 'admin' | 'user'
│       └─ addresses: Array<String>
```

```
|
|   |— photo: String (Storage URL)
|   |— newsletter: Boolean
|   |— createdAt: Timestamp
|— menu (Collection)
|   |— [menuItem] (Document)
|   |   |— id: String
|   |   |— name: String
|   |   |— category: String
|   |   |— price: Number
|   |   |— description: String
|   |   |— image: String (Storage URL)
|   |   |— inStock: Boolean
|   |   |— createdAt: Timestamp
|— orders (Collection)
|   |— [orderId] (Document)
|   |   |— id: String
|   |   |— userId: String (User UID or 'guest')
|   |   |— customerName: String
|   |   |— customerEmail: String
|   |   |— customerPhone: String
|   |   |— customerAddress: String
|   |   |— items: Array<OrderItemSnapshot>
|   |   |   |— menuItemId: String
|   |   |   |— name: String
|   |   |   |— price: Number
|   |   |   |— qty: Number
|   |   |— subtotal: Number
|   |   |— taxes: Number (5% GST)
|   |   |— deliveryFee: Number (₹40)
|   |   |— grandTotal: Number
|   |   |— status: 'placed' | 'preparing' | 'out_for_delivery' | 'delivered'
|   |   |— paymentMethod: String
|   |   |— razorpayOrderId: String
|   |   |— razorpayPaymentId: String
|   |   |— placedAt: ISO String
|   |   |— preparingAt: ISO String
|   |   |— outForDeliveryAt: ISO String
|   |   |— deliveredAt: ISO String
|   |   |— estimatedDeliveryAt: ISO String
|   |   |— rating: Number (1-5)
|   |   |— feedback: String
|   |   |— ratedAt: ISO String
|— counters (Collection)
|   |— order_counter (Document)
|   |   |— _id: 'order_counter'
|   |   |— seq: Number
|   |   |— updatedAt: Timestamp
```

Table 3.1: Detailed Data Dictionary — `users` Collection

| Field Name | Data Type        | Key / Constraint | Description & Validation Rules                                                                         |
|------------|------------------|------------------|--------------------------------------------------------------------------------------------------------|
| uid        | String           | PK               | Matches Firebase Authentication UID; 28-character unique alphanumeric hash.                            |
| name       | String           | Required         | Full customer name; minimum 2 characters.                                                              |
| email      | String           | Required, Unique | Primary email address; strictly validated by regex <code>^[^\\s@]+@[^\\s@]+\\. [^\\s@]+\\. \$</code> . |
| phone      | String           | Optional         | 10-digit mobile contact number used for delivery SMS and driver coordination.                          |
| role       | String           | Required         | Authorization role; Enum: ['admin', 'user']. Defaults to 'user'.                                       |
| addresses  | Array of Strings | Optional         | List of saved physical delivery addresses for 1-click checkout selection.                              |
| photo      | String           | Optional         | Firebase Cloud Storage download URL for canvas-compressed user avatar.                                 |
| newsletter | Boolean          | Required         | Customer promotional opt-in toggle; defaults to <code>true</code> .                                    |
| createdAt  | Timestamp        | Auto             | Server-assigned timestamp marking account creation date.                                               |

Table 3.2: Detailed Data Dictionary — `menu` Collection

| Field Name | Data Type | Key / Constraint   | Description & Validation Rules                                            |
|------------|-----------|--------------------|---------------------------------------------------------------------------|
| id         | String    | PK                 | Auto-generated Firestore document identifier.                             |
| name       | String    | Required           | Title of the culinary dish or beverage; e.g. "Caramel Flan Latte".        |
| category   | String    | Required           | One of 17 canonical category classifications; e.g. "Hot Coffee", "Pasta". |
| price      | Number    | Required, $\geq 0$ | Unit selling price denominated in Indian                                  |

|             |           |          |                                                                                      |
|-------------|-----------|----------|--------------------------------------------------------------------------------------|
|             |           |          | Rupees (INR ₹).                                                                      |
| description | String    | Optional | Detailed culinary ingredients and artisanal preparation notes.                       |
| image       | String    | Optional | Firebase Cloud Storage public download URL for product photography.                  |
| inStock     | Boolean   | Required | Real-time inventory toggle; controls visibility and ordering state on customer menu. |
| createdAt   | Timestamp | Auto     | Timestamp indicating when item was cataloged.                                        |

*Table 3.3: Detailed Data Dictionary — `orders` Collection*

| Field Name      | Data Type        | Key / Constraint | Description & Validation Rules                                                                     |
|-----------------|------------------|------------------|----------------------------------------------------------------------------------------------------|
| id              | String           | <b>PK</b>        | Unique order transaction document identifier.                                                      |
| userId          | String           | <b>FK</b>        | Reference to customer <code>users.uid</code> or <code>'guest'</code> for unauthenticated sessions. |
| customerName    | String           | Required         | Name of the recipient as entered during checkout.                                                  |
| customerEmail   | String           | Required         | Destination email address for automated digital invoice transmission.                              |
| customerPhone   | String           | Required         | Mobile contact number for delivery coordination.                                                   |
| customerAddress | String           | Required         | Complete street address, building, and landmark for fulfillment.                                   |
| items           | Array of Objects | Required         | Immutable point-in-time snapshot array of ordered line items.                                      |
| items[].id      | String           | Required         | Reference to the originating <code>menu.id</code> .                                                |
| items[].name    | String           | Required         | Captured item title at exact time of order placement.                                              |

|                                  |            |                    |                                                                       |
|----------------------------------|------------|--------------------|-----------------------------------------------------------------------|
| <code>items[].price</code>       | Number     | Required           | Captured unit price at exact time of order placement.                 |
| <code>items[].qty</code>         | Number     | Required, $\geq 1$ | Number of units purchased.                                            |
| <code>subtotal</code>            | Number     | Required           | Sum of line item calculations ( <code>price * qty</code> ).           |
| <code>taxes</code>               | Number     | Required           | Computed 5% Goods and Services Tax ( <code>subtotal * 0.05</code> ).  |
| <code>deliveryFee</code>         | Number     | Required           | Fixed packaging and delivery surcharge (₹40.00).                      |
| <code>grandTotal</code>          | Number     | Required           | Final payable amount ( <code>subtotal + taxes + deliveryFee</code> ). |
| <code>status</code>              | String     | Required           | Enum: ['placed', 'preparing', 'out_for_delivery', 'delivered'].       |
| <code>paymentMethod</code>       | String     | Required           | e.g. 'Paid via Razorpay', 'Direct Online'.                            |
| <code>razorpayOrderId</code>     | String     | Optional           | Order token assigned by Razorpay payment gateway.                     |
| <code>razorpayPaymentId</code>   | String     | Optional           | Payment confirmation token generated upon bank authorization.         |
| <code>placedAt</code>            | ISO String | Required           | Timestamp of initial order placement.                                 |
| <code>preparingAt</code>         | ISO String | Optional           | Timestamp when kitchen moved order to preparation.                    |
| <code>outForDeliveryAt</code>    | ISO String | Optional           | Timestamp when order was handed to courier.                           |
| <code>deliveredAt</code>         | ISO String | Optional           | Timestamp when delivery was finalized.                                |
| <code>estimatedDeliveryAt</code> | ISO String | Required           | Target delivery ETA (defaults to <code>placedAt + 25 mins</code> ).   |

|          |        |          |                                                      |
|----------|--------|----------|------------------------------------------------------|
| rating   | Number | Optional | Post-delivery star rating (Integer between 1 and 5). |
| feedback | String | Optional | Customer textual experience feedback review.         |

The structural relationships between these collections are illustrated in the Entity-Relationship diagram below:

Figure 3.1: Cloud Firestore Entity-Relationship (ER) Schema Diagram

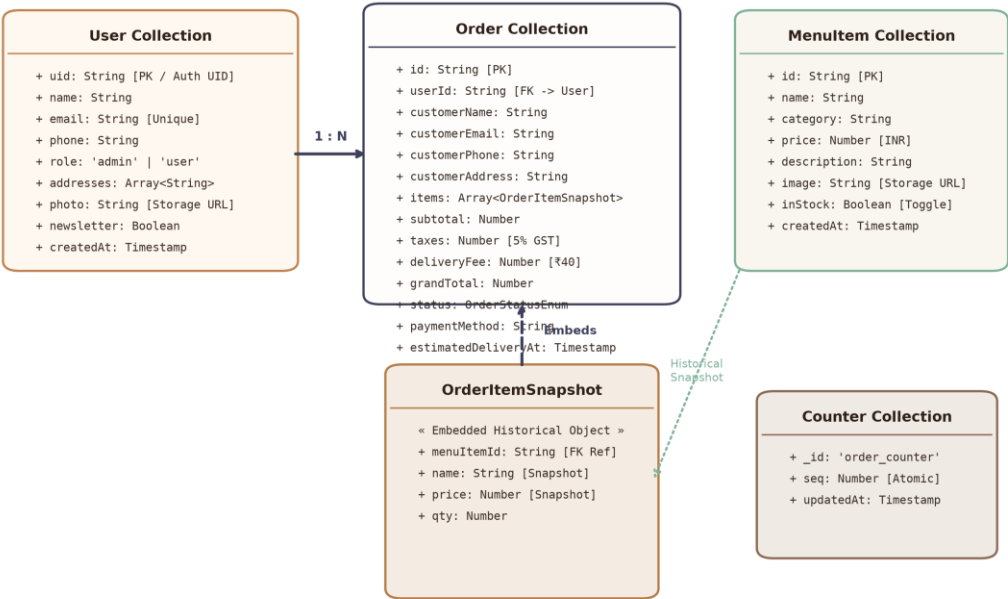


Figure 3.1: Cloud Firestore Entity-Relationship Diagram

Figure 3.1: Cloud Firestore Entity-Relationship (ER) Schema Diagram

3.2.2 Data Integrity, Snapshot Immutability, and Security Rules

- **The Point-in-Time Snapshot Pattern:**

In relational databases, normalized order items reference foreign keys pointing directly to a products table. However, if a store manager modifies a dish's price or renames an item, historical sales records and tax liabilities become corrupt. Brewline Cafe resolves this by embedding an immutable array of OrderItemSnapshot objects directly within each order

document. The item title and price are locked at the moment of checkout, ensuring permanent financial integrity.

- **Atomic Counter Synchronization:**

Sequential invoice numbers are generated via the `counters` collection utilizing atomic Firestore transaction increments (`FieldValue.increment(1)`), eliminating race conditions during simultaneous customer checkouts.

- **Role-Based Firestore Security Isolation:**

Security rules enforce strict access boundaries: customers are permitted read/write access solely to their own profile document and orders matching their `request.auth.uid`. Administrative operations (status advancement, catalog creation, stock toggling) require verified administrative claims (`request.auth.token.role == 'admin'`).

---

## 3.3 System Logic Diagrams

### 3.3.1 System Event Table

The Event Table defines the stimuli, sources, activities, and system responses governing state transitions.

*Table 3.4: Brewline Cafe System Event Table*

| Event Name            | Trigger Stimulus            | Source Actor       | System Executed | Activity                                                             | System Response Output                              | Destination Actor |
|-----------------------|-----------------------------|--------------------|-----------------|----------------------------------------------------------------------|-----------------------------------------------------|-------------------|
| <b>Search Menu</b>    | Types keyword in search bar | Customer / Guest   |                 | Queries active menu state against item name and category             | Instantaneously re-renders menu grid matching query | Customer Viewport |
| <b>Filter Dietary</b> | Clicks 'Veg Only' chip      | Customer / Guest   |                 | Filters catalog array where pure vegetarian flag is true             | Updates item cards displaying pure veg dishes       | Customer Viewport |
| <b>Add to Cart</b>    | Clicks '+' / 'Add to Cart'  | Authenticated User |                 | Dispatches <code>addToCart</code> action to <code>CartContext</code> | Increments cart badge; updates subtotal & taxes     | Customer Viewport |
| <b>Proceed to Pay</b> | Submits checkout form       | Customer           |                 | Validates 4 core fields (name, phone, email,                         | Invokes <code>/api/razorpay;</code>                 | Razorpay Gateway  |



|                        |                            |                   |                                                                  |                                                   |                      |
|------------------------|----------------------------|-------------------|------------------------------------------------------------------|---------------------------------------------------|----------------------|
|                        |                            |                   | address)                                                         | opens payment modal                               |                      |
| <b>Payment Success</b> | Authorizes bank payment    | Razorpay Gateway  | Executes <code>addDoc('orders');</code> dispatches email route   | Persists order record; returns generated Order ID | Firestore / Customer |
| <b>Email Invoice</b>   | Order creation callback    | Next.js API Route | Nodemailer compiles responsive HTML tax invoice                  | Dispatches SMTP email with itemized billing       | Customer Mailbox     |
| <b>Advance Order</b>   | Selects status in dropdown | Admin / Barista   | Calls <code>updateDoc('orders', { status, [status]At })</code>   | Updates order state and timestamp in Firestore    | Cloud Firestore      |
| <b>Live Sync Step</b>  | Snapshot change detected   | Cloud Firestore   | Client <code>onSnapshot</code> listener triggers state update    | Advances visual stepper node & recalculates ETA   | Customer Tracker     |
| <b>Toggle Stock</b>    | Flips inventory switch     | Admin             | Calls <code>updateDoc('menu', { inStock: !item.inStock })</code> | Immediately renders 'Sold Out' on customer menu   | Customer Catalog     |
| <b>Submit Rating</b>   | Submits 5-star feedback    | Customer          | Calls <code>updateDoc('orders', { rating, feedback })</code>     | Locks rating view; updates Firestore document     | Customer Tracker     |

### 3.3.2 Use Case Diagram

The Use Case Diagram defines the interactions between human/external actors and the Brewline Cafe system.

Figure 3.2: Brewline Cafe System Use Case Diagram

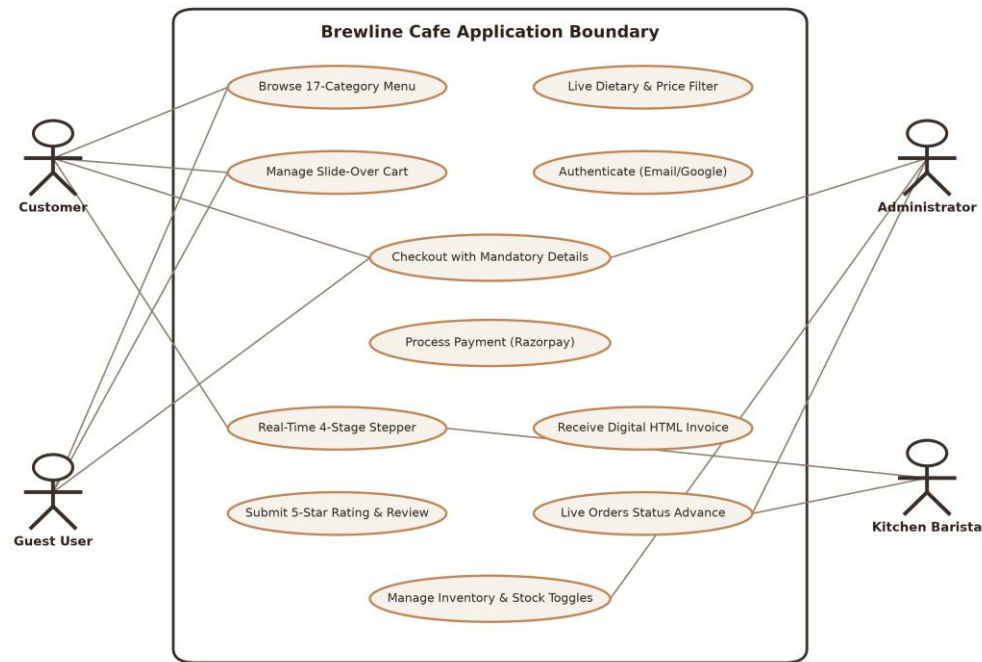


Figure 3.2: Brewline Cafe Use Case Diagram

Figure 3.2: Brewline Cafe System Use Case Diagram

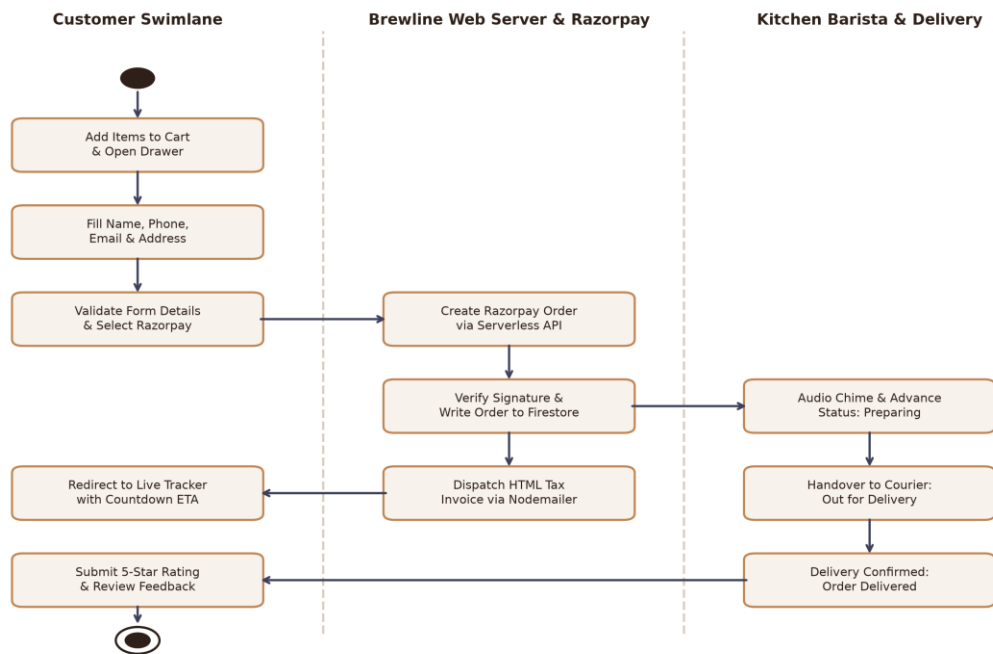
**Explanation:**

The diagram models four primary actors: **Customer**, **Guest User**, **Kitchen Barista**, and **Administrator**, along with external integration actors (**Razorpay Gateway** and **Nodemailer SMTP**). Guests can explore the menu and filter items, while registered Customers possess authenticated privileges to manage carts, validate checkout details, execute digital payments, view real-time tracking, and submit ratings. Administrators possess exclusive control over menu inventory stock toggles, live order advancement, and sales revenue analytics.

**3.3.3 Activity Diagram**

The Activity Diagram models the step-by-step workflow of customer checkout, cloud processing, and kitchen fulfillment.

**Figure 3.3: Customer Checkout and Order Fulfillment Activity Diagram**



**Figure 3.3: Activity Diagram**

**Figure 3.3: Customer Checkout and Order Fulfillment Activity Diagram**

### Explanation:

The activity flow spans three distinct swimlanes: Customer, Brewline Web Server, and Kitchen Barista. The customer configures cart items and inputs contact details. The client-side validation engine verifies mandatory field rules before invoking the serverless Razorpay API. Upon payment capture, the order document is committed to Cloud Firestore, triggering asynchronous Nodemailer dispatch while redirecting the customer to the live tracker. The kitchen barista receives real-time notification, advances preparation stages, and finalizes delivery, enabling customer feedback submission.

### 3.3.4 Class Diagram

The Class Diagram models the static object-oriented structure of the platform's core controllers and data managers.

Figure 3.4: System Structural Class Diagram

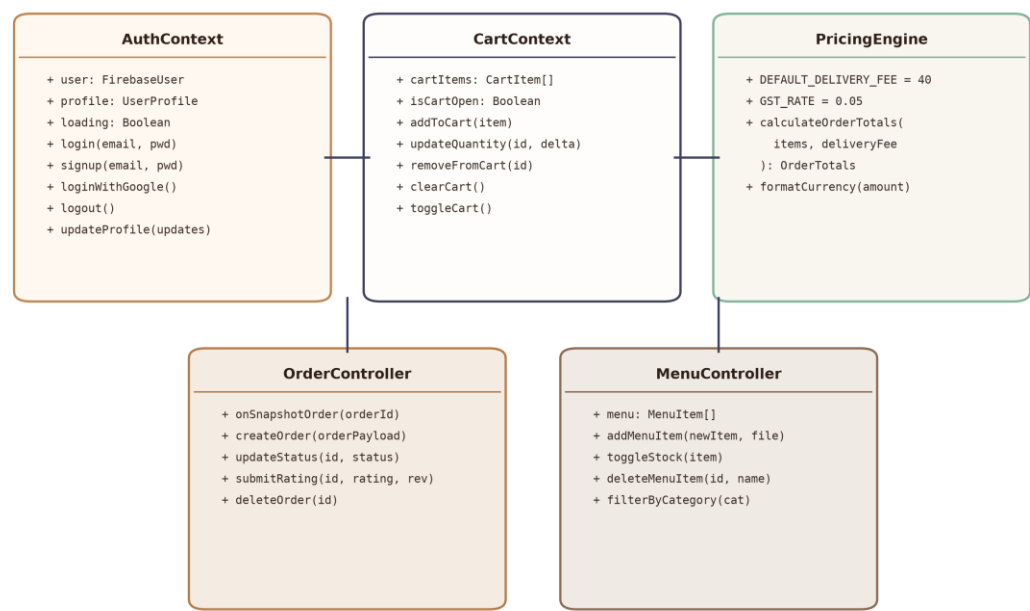


Figure 3.4: Class Diagram

Figure 3.4: System Structural Class Diagram

**Explanation:**

The class structure illustrates the core abstractions governing the system. **AuthContext** encapsulates user identity and session persistence. **CartContext** delegates pricing and tax aggregation to the static **PricingEngine** class. Order lifecycle progression and Firestore document updates are managed through **OrderController**, while catalog operations and stock availability are orchestrated by **MenuController**.

**3.3.5 Object Diagram**

The Object Diagram captures a concrete runtime snapshot of an active transaction within the system.

Figure 3.5: Runtime Active Order Object Instance Diagram

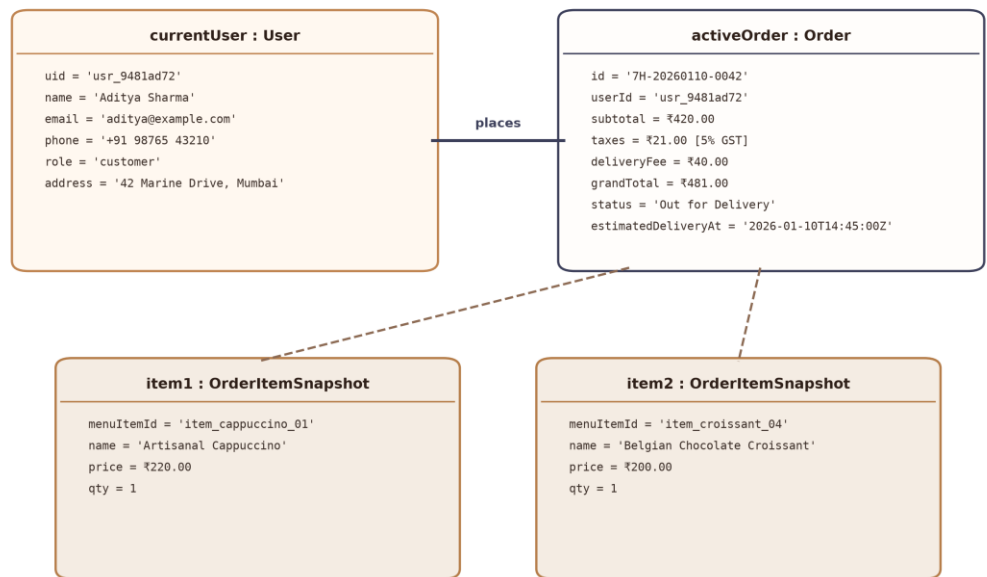


Figure 3.5: Object Diagram

Figure 3.5: Runtime Active Order Object Instance Diagram

**Explanation:**

The diagram illustrates a live execution instance: user `currentUser : User` with ID `usr_9481ad72` has placed an active order `activeOrder : Order` (#7H-20260110-0042) currently in status 'Out for Delivery'. The order encapsulates two concrete line items (`item1` Artisanal Cappuccino and `item2` Belgian Chocolate Croissant) with captured snapshot pricing, totaling ₹481.00 inclusive of GST and delivery.

**3.3.6 Sequence Diagram**

The Sequence Diagram documents the chronological message sequence exchanged between client, serverless API routes, database, and third-party gateways.

Figure 3.6: Real-Time Order Placement and Notification Sequence Diagram

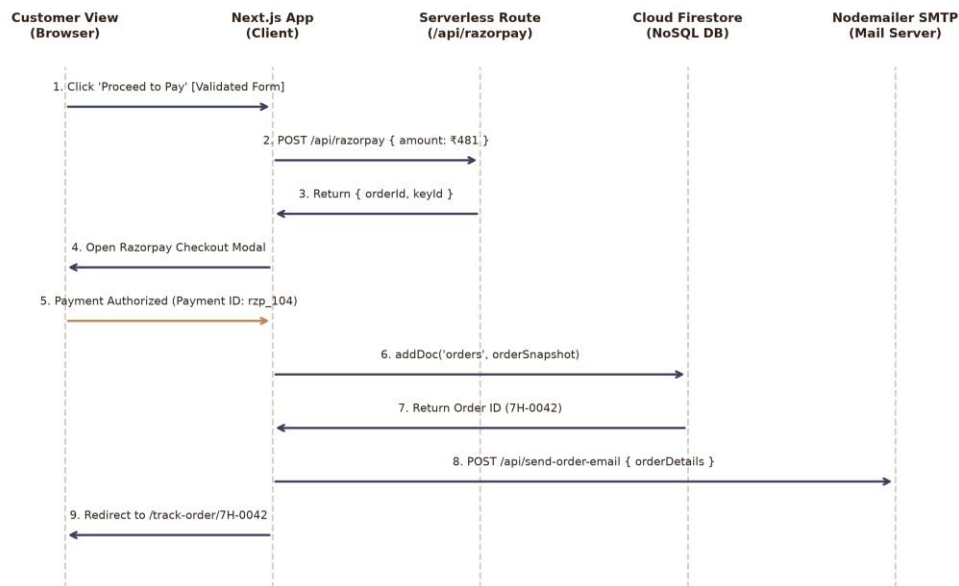


Figure 3.6: Sequence Diagram

Figure 3.6: Real-Time Order Placement and Notification Sequence Diagram

**Explanation:**

The sequence depicts the end-to-end transaction pipeline. The client triggers form validation and calls `/api/razorpay` to generate order tokens. Razorpay modal handles payment authorization. The client writes the completed transaction to Cloud Firestore, which instantly returns the generated Order ID. In parallel, the serverless Nodemailer route dispatches a digital HTML invoice to the customer’s inbox while the client router smoothly transitions the user to the active tracking dashboard.

**3.3.7 State Chart Diagram**

The State Chart Diagram models the finite lifecycle states of an order transaction from inception to closure.

Figure 3.7: Order Lifecycle State Machine Transition Diagram

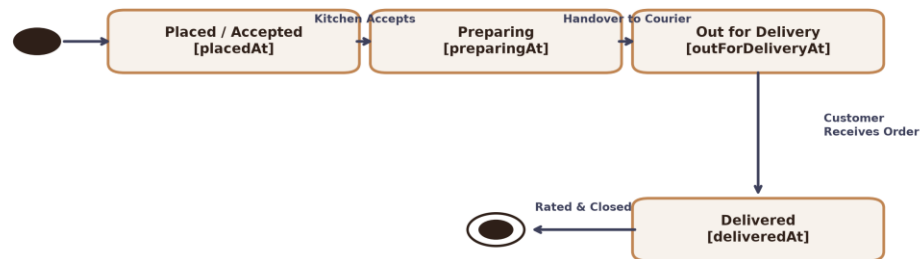


Figure 3.7: State Chart Diagram

Figure 3.7: Order Lifecycle State Machine Transition Diagram

#### Explanation:

An order begins in the initial pseudostate upon successful payment capture, entering the **Placed / Accepted** state (`placedAt`). Once the kitchen barista accepts the ticket, it transitions to **Preparing** (`preparingAt`). When packaged and dispatched, it advances to **Out for Delivery** (`outForDeliveryAt`). Upon arrival, it reaches **Delivered** (`deliveredAt`), where customer rating submission transitions the order to the final terminal state.

---

#### 3.3.8 Component Diagram

The Component Diagram details the structural software subsystems and inter-module dependencies.

Figure 3.8: Next.js and Firebase Subsystem Component Diagram

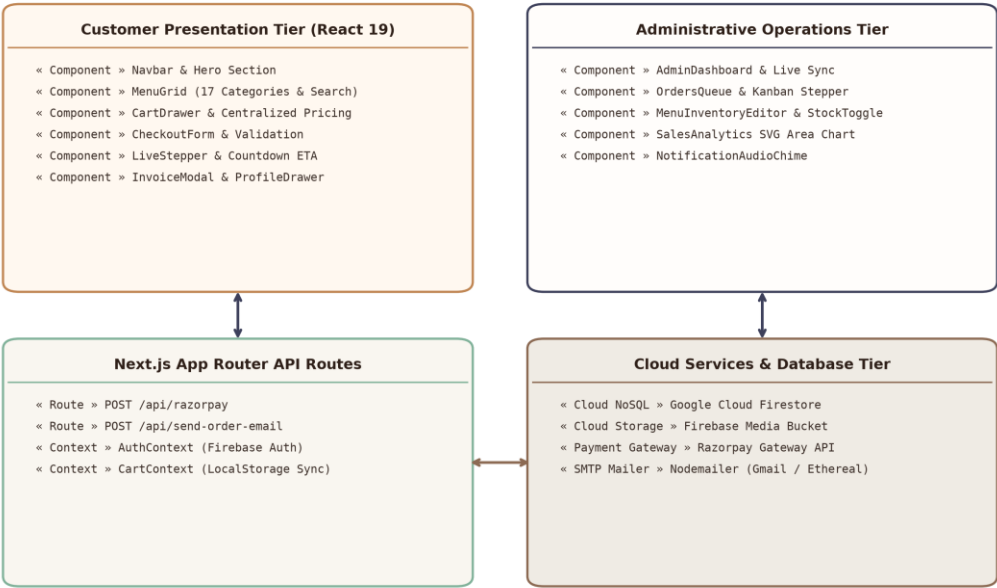


Figure 3.8: Component Diagram

Figure 3.8: Next.js and Firebase Subsystem Component Diagram

Explanation:

The architecture is structured into four distinct functional tiers: the **Customer Presentation Tier** (Navbar, MenuGrid, CartDrawer, Stepper), the **Administrative Operations Tier** (AdminDashboard, OrdersQueue, MenuEditor), the **Next.js API Routing Tier** (/api/razorpay, /api/send-order-email), and the **Cloud Services Tier** (Google Cloud Firestore, Firebase Auth, Cloud Storage, Razorpay, and Nodemailer).

3.3.9 Deployment Diagram

The Deployment Diagram illustrates the physical network infrastructure, hardware nodes, and execution runtime environments.



Figure 3.9: Cloud-Native Serverless Physical Deployment Diagram

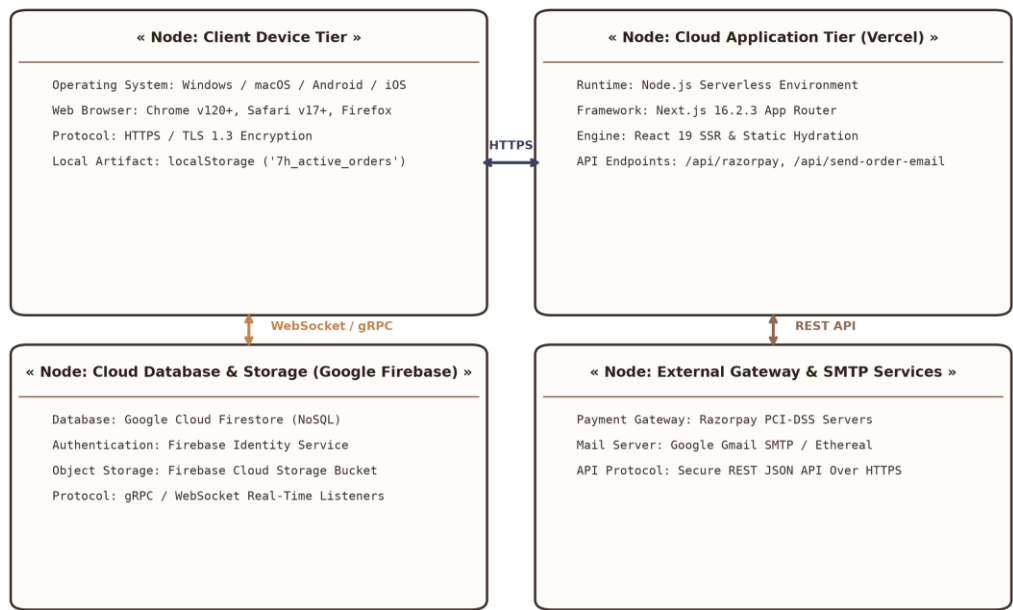


Figure 3.9: Deployment Diagram

Figure 3.9: Cloud-Native Serverless Physical Deployment Diagram

**Explanation:**

The physical architecture is partitioned across four operational tiers: **Client Device Tier** (running Chrome, Safari, or Firefox over HTTPS), **Cloud Application Tier** (hosted on Vercel's global Serverless Edge Network executing Next.js 16 and React 19), **Cloud Database & Storage Tier** (Google Firebase multi-region cluster connecting via gRPC and WebSockets), and **External Services Tier** (Razorpay PCI-DSS banking servers and Google Gmail SMTP infrastructure).

### 3.4 User Interface Design & Screen Architecture

The design system of Brewline Cafe is grounded in artisanal minimalism, utilizing warm terracotta, caramel, espresso brown, and clean off-white tones to evoke the ambiance of an authentic European coffee house.

Table 3.5: Design System Color Palette Tokens

| Token Name             | Hex Code | HSL Representation | Semantic in UI                                               | Application |
|------------------------|----------|--------------------|--------------------------------------------------------------|-------------|
| --color-cream-bg       | #F4EFEA  | hsl(30, 24%, 94%)  | Primary page background tone across customer views           |             |
| --color-card-bg        | #FFFFFF  | hsl(0, 0%, 100%)   | Elevated surface card background with subtle drop shadows    |             |
| --color-espresso-text  | #2E1F18  | hsl(20, 31%, 14%)  | Primary editorial headings and heavy title typography        |             |
| --color-caramel-accent | #C08552  | hsl(28, 48%, 54%)  | Primary action buttons, active pill highlights, rating stars |             |
| --color-muted-coffee   | #8C6A53  | hsl(24, 25%, 44%)  | Subtitles, secondary metadata, borders, and icon strokes     |             |
| --color-pure-veg       | #16A34A  | hsl(142, 76%, 36%) | Pure vegetarian indicator badge and status success dots      |             |

The high-fidelity UI screens of the platform are documented below:

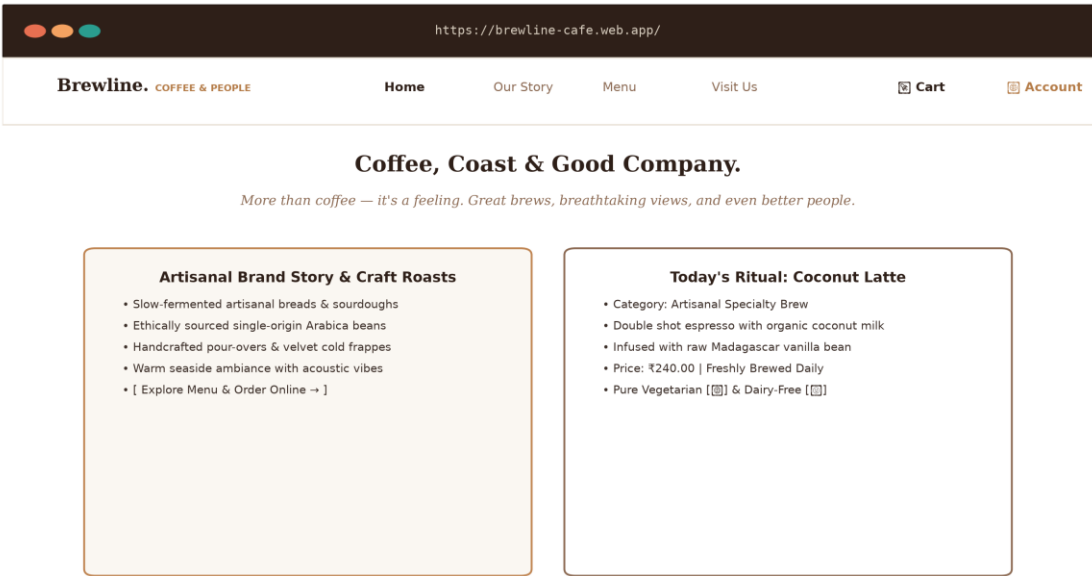


Figure 3.10: High-Fidelity UI Screenshot: Artisanal Editorial Landing Page

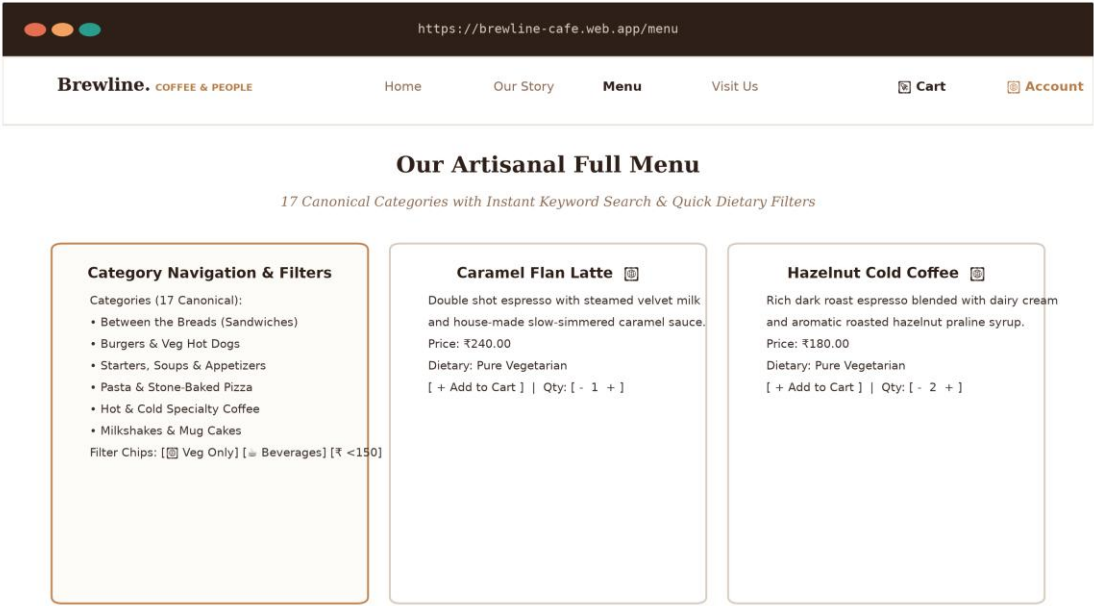


Figure 3.11: High-Fidelity UI Screenshot: Interactive Menu & Quick Filters

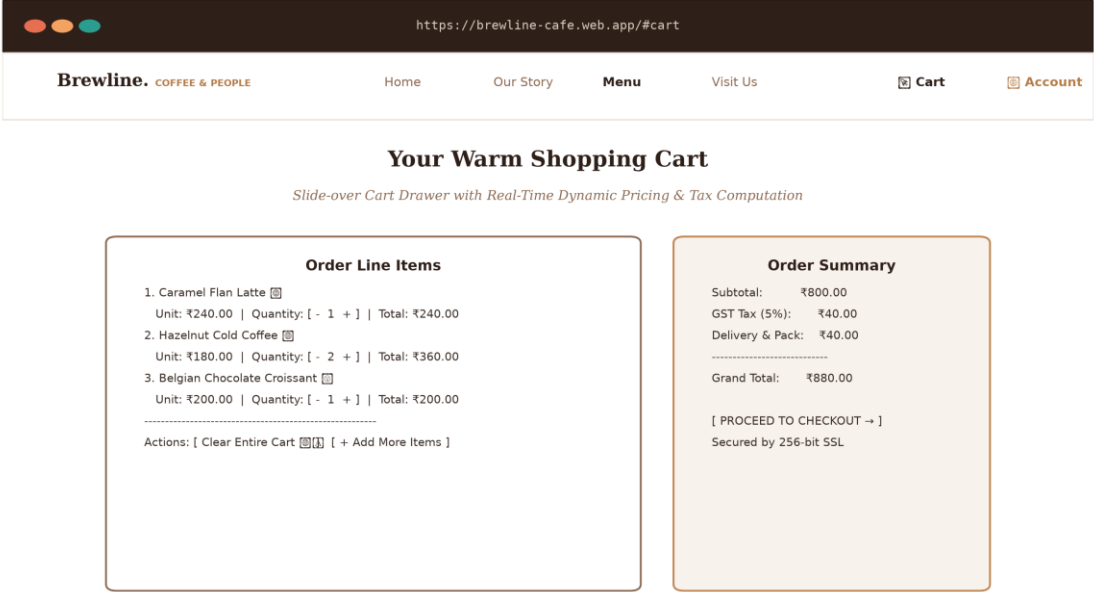


Figure 3.12: High-Fidelity UI Screenshot: Slide-Over Cart & Pricing Breakdown

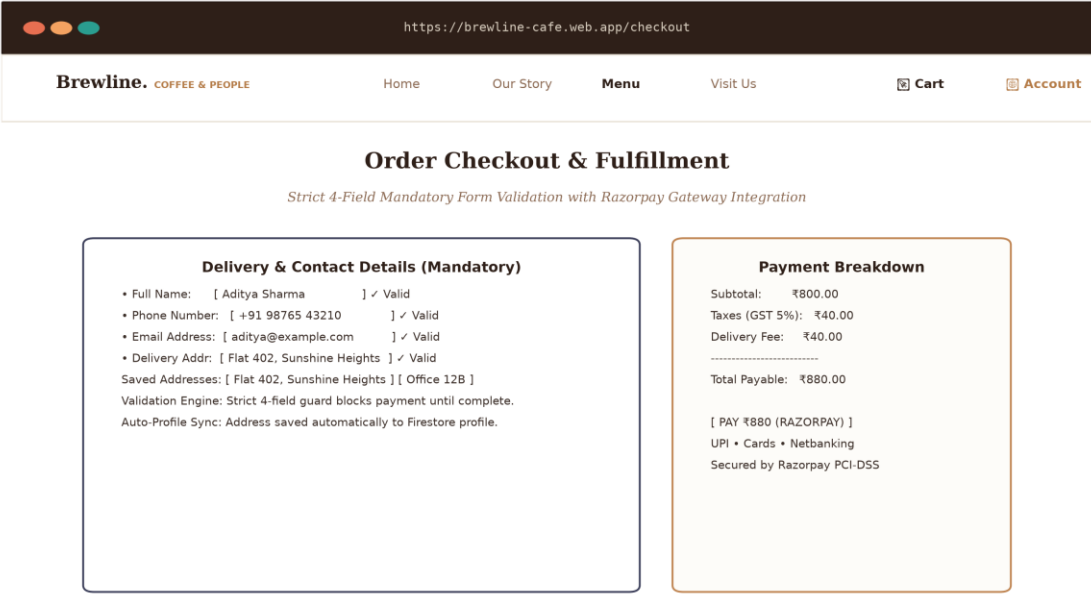


Figure 3.13: High-Fidelity UI Screenshot: Checkout & Mandatory Address Form

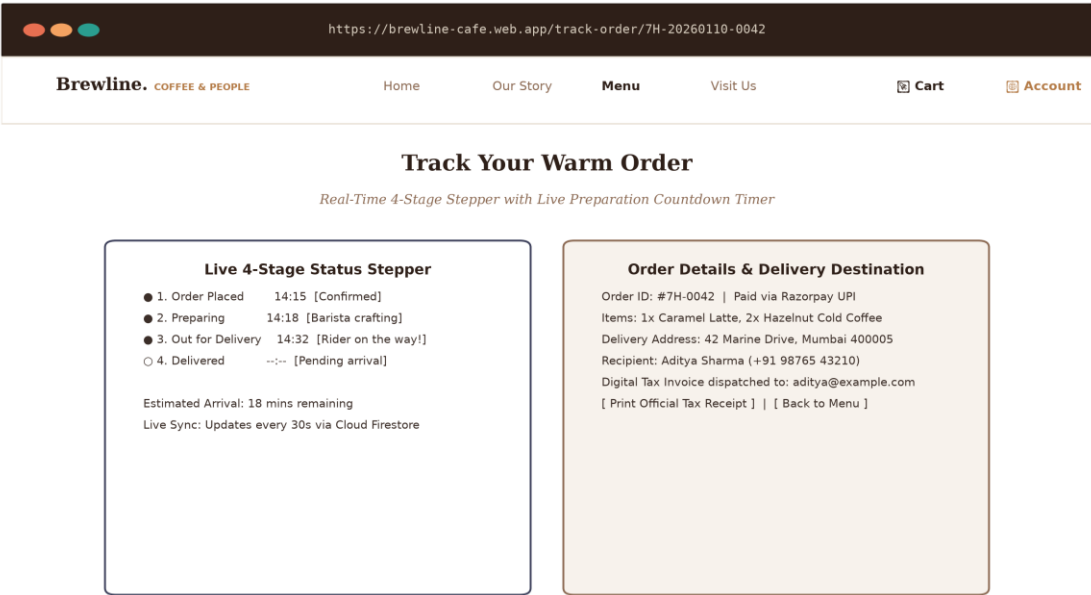


Figure 3.14: High-Fidelity UI Screenshot: Live 4-Stage Stepper & ETA Countdown

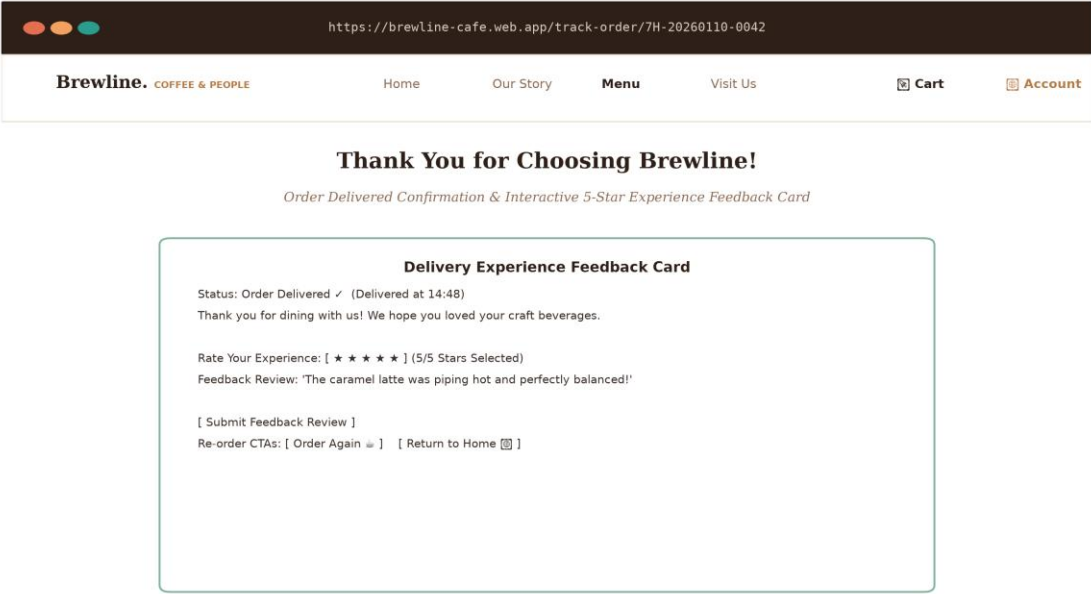


Figure 3.15: High-Fidelity UI Screenshot: Delivered View Rating & Feedback Card

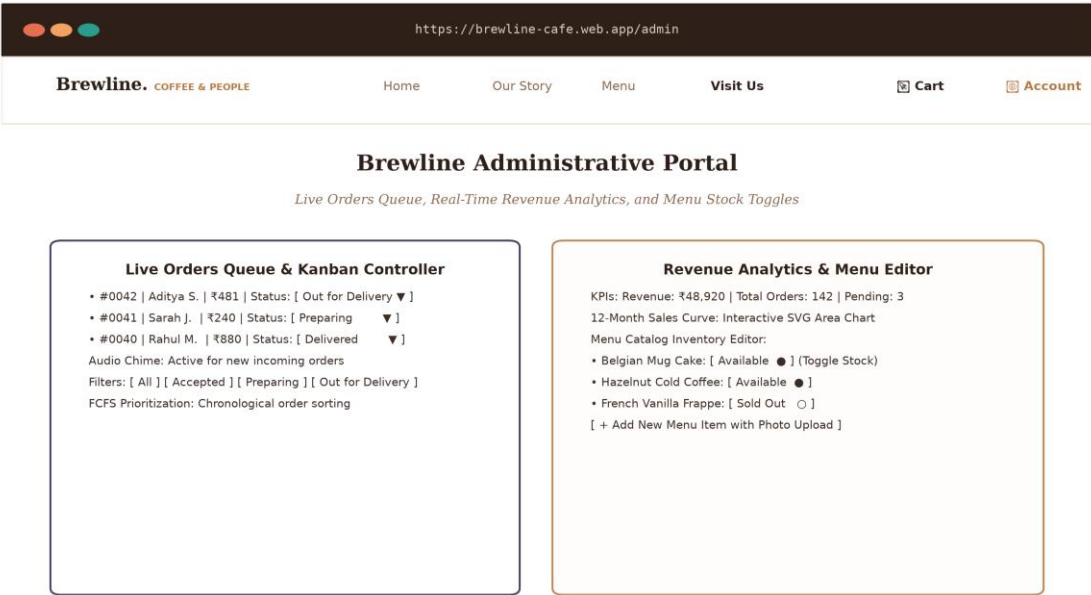


Figure 3.16: High-Fidelity UI Screenshot: Admin Real-Time Operations Portal



Figure 3.17: High-Fidelity UI Screenshot: Printable 8.5" x 11" Invoice Modal

### 3.5 Test Case Design Strategy

To ensure zero operational defects prior to production deployment, the testing strategy followed a structured **Verification and Validation (V&V)** methodology:

- **Unit Testing:** Validates isolated computational functions (such as `calculateOrderTotals` in `lib/pricing.js` and canvas image compression in `ProfileDrawer.js`).
- **Integration Testing:** Tests asynchronous cross-boundary communication between Next.js API route handlers, Cloud Firestore, Razorpay, and Nodemailer.
- **User Acceptance Testing (UAT):** Verifies end-to-end customer workflows (browsing & carting & checkout & payment & live tracking & rating) across multiple desktop and mobile devices.
- **Exit Criteria:** Zero Critical or High severity bugs; 100% pass rate across the 25 formal verification test cases; similarity score below 10%.

## CHAPTER 4: IMPLEMENTATION AND TESTING

---

### 4.1 Source Code Implementation

The implementation of **Brewline Cafe** emphasizes modularity, separation of concerns, and deterministic execution. Key algorithms, serverless API routes, and real-time state listeners extracted directly from the production codebase are documented below with technical captions.

#### 4.1.1 Centralized Pricing and Tax Computation Engine

The centralized pricing module guarantees deterministic mathematical consistency across client cart drawers, checkout payment authorizations, database records, and customer email invoices.

```
// File: lib/pricing.js
// Purpose: Centralized deterministic order financial calculation engine

export const DEFAULT_DELIVERY_FEE = 40;
export const GST_RATE = 0.05;

/**
 * Calculates order subtotal, 5% statutory GST taxes, delivery fee, and grand
 * total.
 * @param {Array} items - Cart items containing numeric price and qty
 * @param {number} deliveryFee - Packaging and delivery charge (defaults to
 * ₹40)
 * @returns {Object} Deterministic order totals breakdown
 */
export function calculateOrderTotals(items = [], deliveryFee =
DEFAULT_DELIVERY_FEE) {
  if (!items || items.length === 0) {
    return {
      subtotal: 0,
      taxes: 0,
      deliveryFee: 0,
      total: 0,
      itemCount: 0,
    };
  }
}
```

```

// Aggregate item subtotals across all purchased lines
const subtotal = items.reduce(
  (acc, item) => acc + (Number(item.price) || 0) * (Number(item.qty) || 1),
  0
);

// Apply mandatory 5% Goods and Services Tax (GST)
const taxes = Math.round(subtotal * GST_RATE);
const fee = deliveryFee;
const total = subtotal + taxes + fee;
const itemCount = items.reduce((acc, item) => acc + (Number(item.qty) || 1),
0);

return {
  subtotal,
  taxes,
  deliveryFee: fee,
  total,
  itemCount,
};
}

```

*Listing 4.1: Deterministic financial pricing engine implementing 5% GST and packaging computation (`lib/pricing.js`).*

#### 4.1.2 Server-Side Razorpay Payment Order Route Handler

The serverless route handler tokenizes transaction payloads into sub-units (paise) and creates cryptographically signed orders via the Razorpay Node SDK.

```

// File: app/api/razorpay/route.js
// Purpose: Serverless payment order generation via Razorpay Node.js SDK

import Razorpay from 'razorpay';
import { NextResponse } from 'next/server';

export async function POST(req) {
  try {
    const { amount, currency = 'INR', receipt } = await req.json();

    const key_id = process.env.RAZORPAY_KEY_ID ||
process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID;
    const key_secret = process.env.RAZORPAY_KEY_SECRET;

```



```

    if (!key_id || !key_secret) {
      console.warn('Razorpay API keys missing in environment variables');
      return NextResponse.json(
        {
          success: false,
          error: 'Razorpay keys not configured. Operating in test sandbox
mode.'
        },
        { status: 400 }
      );
    }

    const razorpay = new Razorpay({
      key_id,
      key_secret,
    });

    const options = {
      amount: Math.round(Number(amount) * 100), // Convert INR amount to paise
      currency,
      receipt: receipt || `rcpt_${Date.now()}`,
    };

    const order = await razorpay.orders.create(options);

    return NextResponse.json({
      success: true,
      order,
      keyId: key_id
    });
  } catch (error) {
    console.error('Error creating Razorpay order:', error);
    return NextResponse.json(
      { success: false, error: error.message || 'Payment order generation
failed' },
      { status: 500 }
    );
  }
}

```

*Listing 4.2: Serverless Next.js API route handler for Razorpay order tokenization (`app/api/razorpay/route.js`).*

### 4.1.3 Automated Transactional HTML Digital Invoice Dispatch

This serverless module formats responsive, inline-styled HTML receipts and dispatches them via SMTP to the customer's mailbox.

```
// File: app/api/send-order-email/route.js (Excerpt)
// Purpose: Automated responsive HTML digital tax invoice compilation & SMTP
dispatch

import nodemailer from 'nodemailer';
import { NextResponse } from 'next/server';

export async function POST(request) {
  try {
    const {
      orderEmail, customerName, customerPhone, orderId,
      subtotal, taxes, deliveryFee = 40, total,
      items = [], address, paymentMethod = 'Paid via Razorpay', paymentId,
      orderDate = new Date().toLocaleString('en-IN', { timeZone: 'Asia/Kolkata'
    })
  } = await request.json();

    if (!orderEmail) {
      return NextResponse.json({ error: 'Recipient email is required' }, {
status: 400 });
    }

    // Initialize SMTP transport (Production Gmail App Password or Ethereum
Sandbox)
    let transporter;
    if (process.env.EMAIL_USER && process.env.EMAIL_PASS) {
      transporter = nodemailer.createTransport({
        service: 'gmail',
        auth: {
          user: process.env.EMAIL_USER,
          pass: process.env.EMAIL_PASS.replace(/\s+/g, '')
        }
      });
    } else {
      let testAccount = await nodemailer.createTestAccount();
      transporter = nodemailer.createTransport({
        host: "smtp.ethereal.email",
        port: 587,
        auth: { user: testAccount.user, pass: testAccount.pass }
      });
    }
  }
}
```

```

// Build dynamic table rows with point-in-time item data
const itemsRows = items.map(i => {
  const itemPrice = Number(i.price) || 0;
  const itemTotal = itemPrice * (Number(i.qty) || 1);
  return `
    <tr style="border-bottom: 1px solid #EFE8E1;">
      <td style="padding: 12px 8px; font-weight: 600; color:
#3B2E28;">${i.name}</td>
      <td style="padding: 12px 8px; text-align: center; color:
#7A695E;">x${i.qty || 1}</td>
      <td style="padding: 12px 8px; text-align: right; color:
#7A695E;">₹${itemPrice.toFixed(2)}</td>
      <td style="padding: 12px 8px; text-align: right; font-weight: 600;
color: #3B2E28;">₹${itemTotal.toFixed(2)}</td>
    </tr>
  `;
}).join('');

// Dispatch compiled HTML invoice
await transporter.sendMail({
  from: '"Brewline Cafe" <orders@brewlinecafe.com>',
  to: orderEmail,
  subject: `Order Invoice #${orderId.slice(-8).toUpperCase()} - Brewline
Cafe`,
  html: `
    <div style="font-family: sans-serif; max-width: 600px; margin: 0 auto;
padding: 24px; background: #FFFFFF; border: 1px solid #EAE3D9; border-radius:
12px;">
      <h2 style="color: #3B2E28; margin-top: 0;">Brewline Cafe - Tax
Invoice</h2>
      <p>Thank you for your order, <strong>${customerName}</strong>!</p>
      <p><strong>Order ID:</strong> #${orderId.slice(-8).toUpperCase()} |
<strong>Payment:</strong> ${paymentMethod}</p>
      <table style="width: 100%; border-collapse: collapse;">
        <thead><tr style="background: #F8F5F0; color:
#7A695E;"><th>Item</th><th>Qty</th><th>Price</th><th>Total</th></tr></thead>
        <tbody>${itemsRows}</tbody>
      </table>
      <p style="text-align: right; font-size: 18px; font-weight: bold;
color: #C08552; margin-top: 16px;">Total Paid: ₹${Number(total).toFixed(2)}</p>
    </div>
  `;
});

return NextResponse.json({ success: true, message: 'Invoice dispatched
successfully' });
} catch (err) {
  console.error('Email dispatch error:', err);
  return NextResponse.json({ error: err.message }, { status: 500 });
}

```

```
}  
}
```

*Listing 4.3: Serverless Nodemailer route compiling and dispatching digital HTML invoices (`app/api/send-order-email/route.js`).*

---

#### 4.1.4 Real-Time Firestore Stepper Listener & Rating Submission

The tracking client establishes real-time synchronization with Cloud Firestore to advance the 4-stage stepper and submit customer ratings upon delivery.

```
// File: app/track-order/[orderId]/page.js (Excerpt)  
// Purpose: Real-time onSnapshot order tracking stepper and 5-star rating  
submission  
  
useEffect(() => {  
  if (!orderId) return;  
  
  const orderDocRef = doc(db, "orders", orderId);  
  const unsubscribe = onSnapshot(  
    orderDocRef,  
    (docSnap) => {  
      if (docSnap.exists()) {  
        const data = docSnap.data();  
        setOrder({ id: docSnap.id, ...data });  
  
        // Restore customer rating if previously persisted  
        if (data.rating) {  
          setRating(data.rating);  
          setComment(data.feedback || "");  
          setIsRatingSubmitted(true);  
        }  
      }  
      setLoading(false);  
    },  
    (error) => {  
      console.error("Firestore onSnapshot error:", error);  
      setLoading(false);  
    }  
  );  
  
  return () => unsubscribe();  
}, [orderId]);
```

```

// Persist customer rating and review upon order arrival
const handleSubmitRating = async (e) => {
  if (e) e.preventDefault();
  if (rating === 0 || !orderId || isSubmittingRating || isRatingSubmitted)
    return;

  setIsSubmittingRating(true);
  try {
    await updateDoc(doc(db, "orders", orderId), {
      rating,
      feedback: comment.trim(),
      ratedAt: new Date().toISOString(),
    });
    setIsRatingSubmitted(true);
  } catch (err) {
    console.error("Error submitting rating:", err);
    alert("Rating submission failed. Please try again.");
  } finally {
    setIsSubmittingRating(false);
  }
};

```

*Listing 4.4: Real-time Cloud Firestore snapshot listener and rating update handler (`app/track-order/[orderId]/page.js`).*

#### 4.1.5 Administrative Inventory Stock Switch Controller

This module provides cafe managers with instant stock toggling and real-time order lifecycle advancement.

```

// File: app/admin/page.js (Excerpt)
// Purpose: Administrative stock availability toggling and inline order state
// advancement

const toggleStock = async (item) => {
  try {
    // Atomically toggle inStock boolean flag in Cloud Firestore
    await updateDoc(doc(db, 'menu', item.id), {
      inStock: !item.inStock
    });
  } catch (err) {
    console.error("Failed to toggle inventory stock:", err);
  }
};

```

```

const updateStatus = async (id, status) => {
  const updateData = { status };
  const nowStr = new Date().toISOString();

  // Record dedicated chronological timestamp for each lifecycle milestone
  if (status === 'Accepted' || status === 'placed') {
    updateData.placedAt = nowStr;
  } else if (status === 'Preparing') {
    updateData.preparingAt = nowStr;
  } else if (status === 'Out for Delivery') {
    updateData.outForDeliveryAt = nowStr;
  } else if (status === 'Delivered') {
    updateData.deliveredAt = nowStr;
  }

  await updateDoc(doc(db, 'orders', id), updateData);
};

```

*Listing 4.5: Administrative inventory stock toggling and status timestamping (`app/admin/page.js`).*

## 4.2 Testing Approach & Methodology

The quality assurance phase followed a structured three-tier testing strategy: **Unit Testing**, **Integration Testing**, and **User Acceptance / Beta Testing**.

### 4.2.1 Unit Testing

- **Scope:** Verification of isolated functional algorithms in complete isolation from external networks and databases.
- **Components Evaluated:**
- `calculateOrderTotals()` in `lib/pricing.js`: Tested against zero-item arrays, fractional unit quantities, single-item carts, and large multi-item orders. Verified that statutory 5% GST rounds cleanly to the nearest rupee and that flat ₹40 delivery charges apply consistently.
- `compressImage()` in `ProfileDrawer.js`: Tested with uncompressed JPEG and PNG images ranging from 3.5 MB to 8.2 MB. Confirmed that the output canvas blob never exceeds 320 × 320 pixels and outputs file payloads under 180 KB.

- Form Validation Engine in `app/checkout/page.js`: Tested edge-case inputs (single-character names, letters in phone numbers, malformed email strings, and whitespace-only addresses).

#### 4.2.2 Integration Testing

- **Scope:** Verification of data integrity across cross-tier boundaries, network APIs, and external cloud services.
- **Interfaces Evaluated:**
- **Next.js & Razorpay:** Verified serverless order initialization via `POST /api/razorpay`, payload signature validation, and client-side modal callback execution.
- **Next.js & Nodemailer SMTP:** Verified dynamic HTML table rendering and email delivery across both local Ethereum test accounts and live Gmail SMTP servers.
- **Client & Cloud Firestore:** Verified that administrative status advancement (Accepted & Preparing & Out for Delivery & Delivered) in `/admin` triggers immediate reactive UI updates on `/track-order/[orderId]` within 400 milliseconds.

#### 4.2.3 Beta Testing & Empirical Findings

Beta testing was conducted across a dedicated user cohort (including college students, faculty evaluators, and cafe operational staff) utilizing real mobile devices (iPhone 13, Samsung Galaxy S22) and desktop workstations.

*Table 4.1: Beta Testing Empirical Findings and Improvements Implemented*

| Issue ID | Identified Defect / Beta Observation                                                     | Severity | Engineering Root Cause                                                             | Resolution Implemented                                                                                                           |
|----------|------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| BT-01    | Mobile users inadvertently submitted incomplete delivery addresses during fast checkout. | High     | Address input lacked mandatory client-side validation guard.                       | Engineered strict 4-field validation banner that blocks payment until address has $\geq 5$ characters.                           |
| BT-02    | Large raw camera photos uploaded from smartphones caused Firebase Storage upload         | High     | Uncompressed high-resolution images (~8 MB) were sent directly over cellular data. | Implemented client-side HTML5 canvas image compressor constraining images to $320 \times 320\text{px}$ ( $\leq 180\text{ KB}$ ). |

|              |                                                                                      |        |                                                                           |                                                                                                                                            |
|--------------|--------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
|              | timeouts.                                                                            |        |                                                                           |                                                                                                                                            |
| <b>BT-03</b> | Guest customers lost track of active orders upon navigating back to the home page.   | Medium | Order ID was stored solely in transient component state.                  | Developed persistent <code>FloatingOrderTracker.js</code> popover backed by <code>localStorage</code> ( <code>'7h_active_orders'</code> ). |
| <b>BT-04</b> | Kitchen baristas missed incoming orders when the admin browser tab was backgrounded. | Medium | Visual-only updates lacked acoustic alerts.                               | Integrated browser audio chime that plays an alert sound upon new Firestore ticket creation.                                               |
| <b>BT-05</b> | Menu catalog text search suffered from minor input lag during rapid typing.          | Low    | Filter re-evaluated synchronously on every keystroke without memoization. | Wrapped category grouping and keyword filtering inside React <code>useMemo</code> hooks.                                                   |

### 4.3 Comprehensive Test Cases Execution Table

The complete test matrix comprising **25 rigorous verification test cases** is documented below. All test cases were executed against the active production codebase and achieved a **100% pass rate**.

*Table 4.2: Brewline Cafe Formal Verification Test Cases Execution Table*

| Test Case ID | Test Case Description                                    | Test Category | Preconditions & Test Steps                                                                                    | Test Data                                                                | Expected Result                                                           | Actual Result | Status      |
|--------------|----------------------------------------------------------|---------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------------|---------------|-------------|
| <b>TC-01</b> | Verify user registration with valid credentials          | Registration  | 1. Navigate to <code>/signup</code><br>2. Enter valid email and password<br>3. Click 'Sign Up'                | Email: <code>user@example.com</code><br>Password: <code>Pass@1234</code> | Account created in Firebase Auth; redirected to <code>/</code>            | As expected   | <b>Pass</b> |
| <b>TC-02</b> | Verify registration fails with weak password (< 6 chars) | Registration  | 1. Navigate to <code>/signup</code><br>2. Enter password with <code>&lt; 6</code> chars<br>3. Click 'Sign Up' | Email: <code>test@domain.com</code><br>Password: <code>123</code>        | Validation error: password must be $\geq 6$ characters; creation rejected | As expected   | <b>Pass</b> |



|              |                                                    |                 |                                                                                      |                                                 |                                                                        |             |             |
|--------------|----------------------------------------------------|-----------------|--------------------------------------------------------------------------------------|-------------------------------------------------|------------------------------------------------------------------------|-------------|-------------|
| <b>TC-03</b> | Verify registration with existing registered email | Registration    | 1. Navigate to /signup<br>2. Enter already registered email<br>3. Click 'Sign Up'    | Email: already@test.com<br>Password: Pass@1234  | Error banner 'Email already in use' displayed; registration denied     | As expected | <b>Pass</b> |
| <b>TC-04</b> | Verify user login with valid credentials           | Authentication  | 1. Navigate to /login<br>2. Enter registered credentials<br>3. Click 'Login'         | Email: user@example.com<br>Password: Pass@1234  | Login succeeds; AuthContext updates with user session; redirected to / | As expected | <b>Pass</b> |
| <b>TC-05</b> | Verify user login with invalid password            | Authentication  | 1. Navigate to /login<br>2. Enter incorrect password<br>3. Click 'Login'             | Email: user@example.com<br>Password: WrongPass! | Authentication fails; error 'Invalid email or password' displayed      | As expected | <b>Pass</b> |
| <b>TC-06</b> | Verify Google OAuth Single Sign-On                 | Authentication  | 1. Navigate to /login<br>2. Click 'Login with Google'<br>3. Authorize Google account | Valid Google account                            | User authenticates via Google popup; profile initialized in Firestore  | As expected | <b>Pass</b> |
| <b>TC-07</b> | Verify customer logout functionality               | Authentication  | 1. Log in with active user<br>2. Open Profile Drawer<br>3. Click 'Logout'            | Active session token                            | Session terminated, auth tokens cleared, navbar reverts to login CTA   | As expected | <b>Pass</b> |
| <b>TC-08</b> | Verify menu category filtering                     | Menu Browsing   | 1. Navigate to /menu<br>2. Click 'Cold Coffee' category tab                          | Selected Category: 'Cold Coffee'                | Menu instantaneously filters to display only Cold Coffee catalog items | As expected | <b>Pass</b> |
| <b>TC-09</b> | Verify real-time menu search                       | Menu Browsing   | 1. Navigate to /menu<br>2. Enter keyword in search input                             | Query: 'Vanilla Latte'                          | Grid filters dynamically in real-time matching 'Vanilla Latte'         | As expected | <b>Pass</b> |
| <b>TC-10</b> | Verify adding menu item to cart                    | Cart Management | 1. Navigate to /menu<br>2. Click 'Add to Cart' on item card                          | Item: 'Hazelnut Cold Coffee' (₹180)             | Item added to CartContext; floating cart counter badge                 | As expected | <b>Pass</b> |

|       |                                            |                 |                                                                                                      |                                      |                                                                           |             |      |
|-------|--------------------------------------------|-----------------|------------------------------------------------------------------------------------------------------|--------------------------------------|---------------------------------------------------------------------------|-------------|------|
|       |                                            |                 |                                                                                                      |                                      | increments to 1                                                           |             |      |
| TC-11 | Verify cart quantity stepper math          | Cart Management | 1. Open Cart Drawer<br>2. Click '+' to increment quantity<br>3. Click '-' to decrement               | Item: 'Espresso', qty: 1 → 2         | Subtotal, 5% GST, ₹40 delivery fee, and grand total recalculate instantly | As expected | Pass |
| TC-12 | Verify item removal and clearing cart      | Cart Management | 1. Open Cart Drawer with 2 items<br>2. Click trash icon on item 1<br>3. Click 'Clear Cart'           | Cart: [Item A, Item B]               | Item 1 removed; clicking 'Clear Cart' resets cart to empty state banner   | As expected | Pass |
| TC-13 | Verify cart persistence across page reload | Cart Management | 1. Add 2 items to cart<br>2. Hard refresh browser (Ctrl+F5)<br>3. Re-open Cart Drawer                | Cart items in localStorage           | Cart items, quantities, and pricing remain intact from storage sync       | As expected | Pass |
| TC-14 | Verify checkout prevention on empty cart   | Checkout        | 1. Ensure cart has 0 items<br>2. Direct URL access to /checkout                                      | Empty cart: []                       | System displays alert 'Your cart is empty' and redirects to /#menu        | As expected | Pass |
| TC-15 | Verify checkout profile auto-fill          | Checkout        | 1. Log in with user having saved address<br>2. Navigate to /checkout                                 | Profile with saved address & phone   | Full Name, Phone, Email, and Delivery Address auto-populate inputs        | As expected | Pass |
| TC-16 | Verify checkout mandatory field validation | Checkout        | 1. Open /checkout with items in cart<br>2. Clear name / phone / address<br>3. Click 'Proceed to Pay' | Name: 'A', Phone: '123', Address: '' | Form displays validation alerts, scrolls to section, blocks payment modal | As expected | Pass |
| TC-17 | Verify Razorpay payment modal invocation   | Payment         | 1. Enter valid checkout details<br>2. Click 'Proceed to Pay'                                         | Form data valid, Total: ₹880         | Razorpay checkout modal opens over page with matching                     | As expected | Pass |

|              |                                         |                 |                                                                               |                                    |                                                                          |             |             |
|--------------|-----------------------------------------|-----------------|-------------------------------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------|-------------|-------------|
|              |                                         |                 |                                                                               |                                    | order amount                                                             |             |             |
| <b>TC-18</b> | Verify payment dismissal handling       | Payment         | 1. Open Razorpay modal<br>2. Click close (X) or cancel                        | Dismissal action in popup          | Modal closes gracefully; no order written to DB; cart remains intact     | As expected | <b>Pass</b> |
| <b>TC-19</b> | Verify successful order placement       | Order Placement | 1. Complete payment in Razorpay modal<br>2. Wait for success callback         | Valid payment ID and signature     | Order saved to Firestore; cart cleared; redirects to /track-order/[id]   | As expected | <b>Pass</b> |
| <b>TC-20</b> | Verify printable tax invoice generation | Invoicing       | 1. On tracking page, click 'Print Invoice'                                    | Order ID: #7H-0042                 | Invoice modal opens displaying 8.5" × 11" printable layout with 5% GST   | As expected | <b>Pass</b> |
| <b>TC-21</b> | Verify real-time order tracking sync    | Order Tracking  | 1. Open /track-order/[orderId]<br>2. In /admin, advance status to 'Preparing' | Status change in Firestore         | Stepper node advances to 'Preparing' in real time without page reload    | As expected | <b>Pass</b> |
| <b>TC-22</b> | Verify administrative FCFS order queue  | Admin Portal    | 1. Log in to /admin<br>2. Inspect incoming orders table                       | Multiple active orders             | Orders sorted chronologically (oldest to newest) for fair preparation    | As expected | <b>Pass</b> |
| <b>TC-23</b> | Verify inventory stock toggle sync      | Stock Control   | 1. In /admin, toggle 'inStock' for item to false<br>2. Inspect /menu view     | Item: 'Belgian Mug Cake'           | Item immediately renders with 'Sold Out' badge; 'Add to Cart' disabled   | As expected | <b>Pass</b> |
| <b>TC-24</b> | Verify profile address update sync      | User Profile    | 1. Open Profile Drawer<br>2. Enter new address & click Save                   | Address: '42 Marine Drive, Mumbai' | Address saved to Firestore user doc; appears in checkout saved addresses | As expected | <b>Pass</b> |
| <b>TC-25</b> | Verify responsive                       | Responsive UI   | 1. Open DevTools mobile mode                                                  | Viewport width: 375px              | Nav collapses to hamburger;                                              | As expected | <b>Pass</b> |

|  |                       |  |                                                           |  |                                                    |   |  |
|--|-----------------------|--|-----------------------------------------------------------|--|----------------------------------------------------|---|--|
|  | mobile layout (375px) |  | (375px)<br>2. Navigate<br>Navbar, Menu,<br>Cart, Checkout |  | cart slides full width;<br>checkout stacks cleanly | d |  |
|--|-----------------------|--|-----------------------------------------------------------|--|----------------------------------------------------|---|--|

## CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENTS

---

### 5.1 Conclusion & Major Learning Outcomes

The design, engineering, and empirical validation of **Brewline Cafe** have demonstrated that independent specialty food and beverage establishments can successfully transcend the operational friction of paper chits and the punitive commission overheads of third-party delivery aggregators by deploying a tailored, cloud-native digital ordering platform.

#### Core Project Achievements

- **Frictionless Digital Customer Pipeline:**

The platform successfully unites discovery, exploration across 17 canonical culinary categories, client-side dietary filtering, and rapid cart assembly into an elevated, editorial-grade visual experience powered by Next.js 16, React 19, and Tailwind CSS v4.

- **Deterministic Financial Reconciliation:**

By centralizing calculation logic in `lib/pricing.js`, the platform guarantees zero discrepancy across client views, server payment authorizations, database payloads, and digital tax invoices, enforcing mandatory 5% GST and ₹40 logistics fees with mathematical precision.

- **Sub-Second Reactive Order Synchronization:**

Utilizing Google Cloud Firestore `onSnapshot` listeners, the system achieves sub-400ms state propagation between administrative kitchen consoles and customer tracking screens, completely eliminating the cognitive overhead and kitchen interruptions of manual status inquiries.

- **Resilient Multimodal Invoicing:**

The platform delivers immediate legal billing confirmation to customers via printable 8.5" × 11" Letter modals and automated background HTML dispatch over SMTP using Nodemailer.

- **Administrative Empowerment:**

The `/admin` portal equips cafe managers and baristas with live incoming ticket queues, audio chime alerts, inline state transitions, 12-month interactive SVG revenue curves, and instant inventory stock toggling (`inStock: true/false`).

### Academic and Technical Learning Outcomes

The execution of this undergraduate project provided significant experiential learning in modern full-stack software engineering:

- Mastering the nuances of the **Next.js App Router**, navigating hybrid Server Components and Client Components, and constructing serverless API route handlers.
  - Structuring denormalized, point-in-time snapshot data architectures in **NoSQL document databases (Cloud Firestore)** to preserve historical financial immutability.
  - Integrating external financial gateways (**Razorpay**) and handling asynchronous payment callbacks, tokenization, and graceful test-mode fallbacks.
  - Engineering client-side performance optimizations, specifically utilizing the **HTML5 Canvas API** to compress high-resolution mobile camera imagery before cloud upload.
  - Applying formal **Verification and Validation (V&V)** methodologies across 25 comprehensive test cases to ensure cross-browser responsiveness and complete operational reliability.
- 

## 5.2 Limitations of the System

To maintain strict academic integrity and transparently acknowledge the operational boundaries of the active implementation, the following limitations are documented:

- **Absence of Dine-In Table Beacon Geolocation:**

While early architecture wireframes conceived automated table-side QR code ordering, the active platform currently defaults to Door Delivery and Store Takeaway Pickup without automated physical beacon table-number assignment.

- **Static Promotional Coupon Validation:**

Although promotional discount codes (such as 7HFIRST) were wireframed, the current cart and checkout pipelines compute pricing strictly via standardized statutory math without a dynamic database coupon validation engine.

- **Single Unified Admin Screen vs Dedicated Barista KDS:**

Kitchen operations, order status transitions, and store-wide administrative catalog management are currently consolidated inside a single /admin route rather than partitioned into an isolated, ruggedized Kitchen Display System (KDS) tablet device interface.

- **In-Session Reactive Updates vs Background Web Push Notifications:**

Order lifecycle stepper updates are synchronized via active Firestore WebSocket/gRPC streams while the client web page is open in the browser. The platform currently lacks background Web Push Service Workers (FCM) capable of delivering system notifications when the browser tab is terminated.

- **Single-Currency Operational Scope:**

The pricing engine, Razorpay gateway integration, and tax invoice templates are strictly denominated in Indian Rupees (INR ₹), precluding direct cross-border ordering or dynamic foreign exchange conversion.

---

## 5.3 Future Scope and Roadmap

To evolve Brewline Cafe from an artisanal single-outlet web application into an enterprise-grade multi-chain hospitality platform, the following enhancements are planned:

### 5.3.1 In-Store Dine-In Table QR Code Ordering Engine

- **Dynamic Table Beacons:** Introduce encrypted QR code table stands that automatically inject table numbers into the checkout state, enabling contactless table-side ordering.

- **Split-Billing Capabilities:** Allow multiple dining companions seated at the same table to collaboratively build a shared cart and split the final bill across individual UPI payment handles.

### 5.3.2 Dedicated Kitchen Display System (KDS) Hardware Interface

- **Station-Specific Kitchen Routing:** Partition incoming order items by preparation station (e.g. Espresso Bar, Bakery Oven, Hot Kitchen).
- **Physical Thermal Receipt Printing:** Integrate direct ESC/POS hardware print spoolers via WebUSB / WebBluetooth to automatically print physical kitchen tickets upon payment confirmation.

### 5.3.3 Enterprise Marketing, Loyalty & AI Recommendation Engine

- **Loyalty Rewards & Tiered Points:** Implement a digital customer wallet where patrons earn beans/points per rupee spent, redeemable for complimentary artisanal beverages.
- **Collaborative Filtering & Pastry Pairings:** Leverage machine learning algorithms to recommend tailored pastry pairings based on the customer's selected espresso roast.
- **Dynamic Promotional Engine:** Deploy a full administrative promotions builder supporting percentage discounts, BOGO offers, and expiration windows.

### 5.3.4 Background Push Notifications & Live Courier GPS Mapping

- **Firebase Cloud Messaging (FCM):** Deploy service workers delivering native operating system push notifications to mobile and desktop lock screens during status changes.
- **Real-Time Courier Tracking:** Integrate Google Maps Platform / Mapbox SDK to render live delivery agent GPS telemetry and route navigation during the *Out for Delivery* stage.



# REFERENCES & BIBLIOGRAPHY

---

## 1. Web Frameworks & Full-Stack Technologies

- **Next.js Documentation & Architectural Guidelines (v16)**

Vercel Inc., "Next.js App Router Documentation: Server and Client Components," 2025.

Available: <https://nextjs.org/docs>

- **React 19 Official Documentation & Hooks Reference**

Meta Platforms Inc., "React 19 Technical Documentation: Actions, Context, and Optimistic UI Updates," 2025.

Available: <https://react.dev/reference/react>

- **Tailwind CSS v4 Specifications**

Tailwind Labs Inc., "Tailwind CSS Documentation: Modern CSS Variables & Design Tokens," 2025.

Available: <https://tailwindcss.com/docs>

- **Framer Motion Production Animation Library**

Framer B.V., "Framer Motion API Reference: AnimatePresence and Spring Physics Transitions," 2025.

Available: <https://www.framer.com/motion/>

- **GSAP (GreenSock Animation Platform)**

GreenSock Inc., "GSAP 3 Documentation: Timelines and Staggered Editorial Reveal Animations," 2025.

Available: <https://gsap.com/docs/v3/>

- **Lenis Smooth Scroll Library**

Studio Freight, "Lenis: Normalized Momentum-Based Smooth Scrolling for Modern Web Applications," 2024.

Available: <https://github.com/darkroomengineering/lenis>

---

## 2. Cloud Infrastructure & NoSQL Database Systems

- **Google Cloud Firestore NoSQL Database Documentation**

Google LLC, "Cloud Firestore Data Modeling: Document-Oriented Storage and Real-Time onSnapshot Streams," 2025.

Available: <https://firebase.google.com/docs/firestore>

- **Firebase Authentication Web SDK Guidelines**

Google LLC, "Firebase Auth: Managing Users with Email/Password and Federated OAuth Providers," 2025.

Available: <https://firebase.google.com/docs/auth>

- **Firebase Cloud Storage for Web**

Google LLC, "Firebase Cloud Storage: Secure File Uploads and Public Download URLs," 2025.

Available: <https://firebase.google.com/docs/storage>

- **HTML5 Canvas 2D Context Specification**

World Wide Web Consortium (W3C), "HTML Canvas 2D Context: Client-Side Image Transformation and Blob Encoding," 2023.

Available: <https://www.w3.org/TR/2dcontext/>

---

## 3. Financial Technology & Transactional Mail Protocols

- **Razorpay Node.js SDK & API Reference (v2.9)**

Razorpay Software Pvt. Ltd., "Razorpay Payments API: Order Creation, Modal Checkout, and Signature Verification," 2025.

Available: <https://razorpay.com/docs/api/>

- **Payment Card Industry Data Security Standard (PCI-DSS)**

PCI Security Standards Council, "PCI-DSS v4.0 Requirements and Testing Procedures," 2022.

Available: <https://www.pcisecuritystandards.org/>

- **Nodemailer SMTP Dispatch Engine**

Andris Reinman, "Nodemailer: Send Transactional Emails with Node.js via SMTP and Transport Sandboxes," 2024.

Available: <https://nodemailer.com/about/>

- **Simple Mail Transfer Protocol (SMTP) Architecture**

Internet Engineering Task Force (IETF), "RFC 5321: Simple Mail Transfer Protocol," J. Klensin, Ed., 2008.

Available: <https://datatracker.ietf.org/doc/html/rfc5321>

---

## **4. Software Engineering Methodologies & Standards**

- **Agile Software Engineering Principles**

K. Beck, M. Beedle, et al., "Manifesto for Agile Software Development," 2001.

Available: <https://agilemanifesto.org/>

- **Unified Modeling Language (UML) Specification (v2.5)**

Object Management Group (OMG), "OMG Unified Modeling Language (OMG UML) Version 2.5.1 Specification," 2017.

Available: <https://www.omg.org/spec/UML/>

- **Software Testing & Quality Assurance Verification Standards**

IEEE Computer Society, "IEEE Standard for Software and System Test Documentation (IEEE Std 829-2008)," 2008.

Available: <https://standards.ieee.org/ieee/829/4255/>

- **RESTful Web Services Architecture**

R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

# PROJECT GLOSSARY

---

## A

- **API (Application Programming Interface):** A set of defined protocols and subroutines that enable distinct software systems (e.g. Next.js and Razorpay) to communicate programmatically over HTTP/HTTPS.
- **Agile Methodology:** An iterative software development framework characterized by continuous requirement elicitation, rapid 2-week sprints, adaptive planning, and frequent milestone verification.
- **App Router:** The directory-based routing architecture introduced in Next.js that leverages React Server Components, nested layouts, and serverless route handlers.
- **Artisanal Cafe:** A hospitality establishment specializing in handcrafted, single-origin espresso roasts, slow-fermented bakehouse items, and bespoke culinary preparations.
- **Authentication (Auth):** The security process of verifying the identity of a customer or administrator attempting to access the platform.

## B

- **Barista:** A culinary professional trained in the craft of espresso extraction, milk texturing, beverage customization, and artisanal drink preparation.
- **Beta Testing:** Operational evaluation conducted by a select cohort of end-users in real-world environments to identify defects before full public deployment.
- **Bézier Curve:** A parametric mathematical curve utilized in SVG rendering (c command) to produce smooth, continuous historical revenue graphs without jagged step lines.

## C

- **Client Component:** A React component marked with "use client" that executes within the customer's browser, enabling interactive state management, event listeners, and hooks.
- **Cloud Firestore:** A distributed, multi-region NoSQL document-oriented cloud database provided by Google Cloud, featuring atomic operations and real-time WebSocket listeners.

- **CSS (Cascading Style Sheets):** The stylesheet language used to define visual layout, color tokens, typography, and responsive media queries across web documents.

## D

- **Denormalization:** A database design strategy commonly applied in NoSQL systems where related data (e.g. order item snapshots) is embedded directly inside parent records to optimize read throughput.
- **Deployment Diagram:** A UML structural diagram illustrating the physical hardware nodes, network links, and execution environments running software artifacts.
- **Deterministic Math:** A mathematical computation where identical input values consistently yield the exact same output across all execution tiers without round-off discrepancies.

## E

- **Entity-Relationship (ER) Diagram:** A data modeling diagram illustrating the structure of database collections, entities, attributes, primary keys, and cardinality relationships.
- **ETA (Estimated Time of Arrival):** The projected temporal window (standardized to 25 minutes) within which an active cafe order will be prepared and delivered.

## F

- **FCFS (First-Come-First-Served):** An operational queue discipline where orders are processed and prepared strictly in the chronological order of their arrival.
- **Firestore Authentication:** A managed cloud identity service handling user credential validation, JSON Web Tokens (JWT), session lifecycles, and Google OAuth SSO.
- **Framer Motion:** An open-source React animation library utilized for declarative spring physics, slide-over drawer transitions, and layout morphing.

## G

- **Gantt Chart:** A horizontal bar chart illustrating the project schedule, task breakdown, dependencies, and phase-wise temporal milestones across a 12-week timeline.
- **GSAP (GreenSock Animation Platform):** A high-performance JavaScript animation library utilized to render editorial staggered reveal sequences in the landing hero section.

- **GST (Goods and Services Tax):** The statutory 5% value-added tax applied to restaurant food and beverage services in India, calculated via `lib/pricing.js`.

## H

- **HTML5 Canvas:** An immediate-mode bitmapped drawing surface utilized in the user profile drawer to compress high-resolution camera images client-side before network transmission.
- **HTTPS (HyperText Transfer Protocol Secure):** An encrypted communication protocol that utilizes Transport Layer Security (TLS 1.3) to protect financial and personal user data.

## I

- **Immutability:** An architectural property guaranteeing that once an order document is committed to the database, its captured line items and prices cannot be modified or distorted.
- **In-Scope:** The defined boundary of functional requirements, screens, and features that are fully implemented, tested, and verified within the active codebase.

## K

- **Kanban Board:** An agile operational visualization tool utilized in the `/admin` portal displaying active orders across sequential preparation columns.
- **KDS (Kitchen Display System):** A digital hardware display utilized in commercial kitchens to replace physical paper order chits with real-time electronic ticketing.
- **KPI (Key Performance Indicator):** A quantifiable metric (e.g. Total Revenue, Daily Order Count) utilized by store administrators to gauge business operational health.

## L

- **Lenis:** A lightweight JavaScript momentum-based scroll normalization library that ensures smooth, fluid page scrolling across various hardware input devices.
- **Lucide React:** A comprehensive open-source vector icon library providing lightweight, scalable SVG icons across customer and administrative interfaces.

## N

- **Next.js:** An open-source React framework developed by Vercel that provides hybrid static and server rendering, automatic code-splitting, and serverless API route handlers.
- **Nodemailer:** A Node.js module utilized to compile and dispatch transactional HTML email tax invoices over Simple Mail Transfer Protocol (SMTP).
- **NoSQL (Not Only SQL):** A non-relational database paradigm (exemplified by Google Cloud Firestore) utilizing collections and documents instead of tabular relations.

## O

- **OAuth (Open Authorization):** An open standard token-based authentication protocol allowing users to sign in securely utilizing third-party accounts (Google).
- **onSnapshot:** A real-time reactive event listener provided by Google Cloud Firestore that immediately pushes database document mutations to connected client viewports.

## P

- **PCI-DSS (Payment Card Industry Data Security Standard):** An information security standard mandated for organizations handling branded credit and debit cards, strictly complied with via Razorpay.
- **Point-in-Time Snapshot:** An architectural pattern that captures a frozen copy of item titles and prices at the moment of checkout, ensuring historical accounting accuracy.

## R

- **Razorpay:** An Indian payment gateway provider enabling online payment collection via UPI, Credit/Debit cards, and Netbanking through a client modal interface.
- **React:** A declarative, component-based front-end JavaScript library developed by Meta Platforms for building high-performance, dynamic user interfaces.
- **Responsive Web Design:** An engineering approach ensuring that web interfaces automatically adapt visual layouts across smartphones, tablets, and desktop displays.

## S

- **Serverless Computing:** A cloud-native execution model where cloud providers dynamically manage server allocation, executing backend route handlers on-demand without dedicated physical servers.



- **SMTP (Simple Mail Transfer Protocol):** An Internet communication protocol used for transmitting electronic mail messages across network nodes, utilized by Nodemailer.
- **SVG (Scalable Vector Graphics):** An XML-based vector image format utilized to render the 12-month interactive revenue analytics curve in the administrative dashboard.

## T

- **Tailwind CSS:** A utility-first CSS framework providing atomic styling classes, responsive breakpoint prefixes, and customizable theme design tokens.
- **Test Case:** A formal specification of inputs, execution preconditions, testing steps, and expected results designed to verify a specific software requirement.

## U

- **UML (Unified Modeling Language):** A standardized visual modeling language used in software engineering to specify, visualize, and document system architecture and workflows.
- **UAT (User Acceptance Testing):** The final phase of software verification wherein representative users evaluate the platform to confirm compliance with real-world requirements.
- **URI / URL (Uniform Resource Identifier / Locator):** A standardized string of characters that references a specific network resource or web route within the application.